Master Thesis

Memory Efficient B*

Pascal R. Anema

Thesis submitted in partial fulfillment
of the requirements for the degree of
Master of Science of Artificial Intelligence
at the Department of Advanced Computing Sciences
of the Maastricht University

Thesis Committee:

Prof. Dr. Mark H. M. Winands
Dr. Dennis J. N. J. Soemers

Maastricht University
Faculty of Science and Engineering
Department of Advanced Computing Sciences

April 8, 2026

Preface

This thesis builds directly on the work from a research internship report (Anema, 2025), which was written under the supervision of Prof. Dr. Mark H.M. Winands and Dr. Dennis J.N.J. Soemers and was submitted on 27 February 2025. The initial goal of that report was to achieve a result now presented in this thesis: a description of the depth-first B* variant df-B*. However, this thesis would not have been the same without the advances in artificial game trees developed during the internship report.

The process of writing this thesis has been an exercise in limiting my creativity. Developing the algorithms and conducting the experiments was engaging and exciting, but the task of documenting the work proved challenging at times. I would like to thank several people for supporting me throughout this process, in particular by providing the encouragement needed to prioritise writing over implementing further algorithms.

I would like to express my gratitude to Prof. Mark H.M. Winands for his supervision and guidance throughout both the internship and this thesis. I want to thank my parents and my siblings for their support in various forms, and my dearest friend for her unwavering support and understanding.

I wish you a pleasant reading of this thesis.

Abstract

Artificial Intelligence (AI) techniques in games have been developed for decades, progressing the field of computer-aided decision-making with them. Game tree search is a particular branch of game AI. This discipline represents the game as a rooted tree, for which the best node at its root is the next move to play. B* is a best-first game-tree search technique developed in the late 1970s, which was later used for competitive endgame play of Scrabble in the 2000s. B* is unique in its construction and capabilities, but has not been extensively researched in the last two decades. A recent report has shown that B* can solve large problems more efficiently than was previously known. However, B* is limited in the size of problems it can solve, due to its memory requirements. In this thesis, we address this problem by introducing two B* variants: the memory-efficient multi-level B^{*2} and the memory-independent depth-first df-B*. Experimental evaluation on over 37,000 artificial game trees showed B^{*2} requires just 7.6 times the square root of the memory usage of B*. Whereas the experiments showed that df-B* only requires a near-constant amount of memory, effectively never running out of memory on any problem. However, we observed that df-B* has significantly reduced computational efficiency by approximately a factor of 22 to 23 compared to B*. By contrast, we found B^{*2} was able to solve some problems more efficiently than B*, and was, on average, only a factor of up to 3 times less computationally efficient. In conclusion, B^{*2} was found to be effective for search on large problems, whereas df-B* was found to be effective for search on very large problems, being effectively unrestrained by memory limitations. These results were found to align with existing research on memory-efficient variants of the best-first game-tree solver Proof-Number Search.

Keywords: B* Search · Artificial Game Trees · Depth-First Search · Adversarial Search · Artificial Intelligence

Contents

Preface **ii**

Abstract **iii**

Contents **iv**

1 Introduction **1**
1.1 Artificial Intelligence and Games . . . 1
1.2 Search and Games . . . 2
1.3 Adversarial Game Solvers . . . 3
1.3.1 Proof-Number Search . . . 4
1.3.2 λ-Search . . . 4
1.3.3 B* Search . . . 5
1.4 Problem Statement and Research Questions . . . 6
1.5 Thesis Outline . . . 6

2 Adversarial Game Solvers **7**
2.1 B* Search . . . 7
2.1.1 Original Description . . . 7
2.1.2 Strategy Selection . . . 10
2.2 Proof-Number Search . . . 11
2.2.1 Original Description . . . 11
2.2.2 PN^2: Search on Larger Trees . . . 12
2.2.3 df-PN: Search on any Size Tree . . . 12

3 Saving Memory for B* **13**
3.1 Additional B* Adjustments . . . 13
3.2 A Multi-Level Approach to B* . . . 15
3.2.1 B^{*2} Search . . . 15
3.2.2 Second-Level Tree-Size Limit . . . 17
3.2.3 Extension to Many-Levelled Search . . . 18
3.3 A Depth-First Approach to B* . . . 21
3.3.1 df-B* Search . . . 21
3.3.2 Challenges . . . 23

4 Artificial Game Trees **26**
4.1 Origin . . . 26
4.2 Description . . . 27
4.2.1 Seeds for Canonical Trees . . . 27
4.2.2 Node Generation . . . 27
4.3 Parameter Overview . . . 29

5 Experiments and Results **30**
5.1 Experimental Setup . . . 30
5.1.1 Hardware Specifications . . . 30
5.1.2 List of Terms . . . 30
5.1.3 Experiment 1: General Comparison . . . 31
5.1.4 Experiment 2: Memory Usage . . . 33
5.2 Results . . . 34
5.2.1 Experiment 1 . . . 34
5.2.2 Experiment 2 . . . 38

6 Conclusions and Future Research **42**
6.1 Research Questions . . . 42
6.2 Problem Statement . . . 44
6.3 Future Research . . . 44

References **46**

A Additional Tables **49**

B Additional Details **51**
B.1 Seed Allocation Experiment 1 . . . 51

C Additional Figures **52**

Chapter 1

Introduction

This thesis covers the topic of game-playing Artificial Intelligence (AI) systems referred to as game-tree search techniques. We focus specifically on a game-solving technique called B* (Berliner, 1979), which efficiently proves the best moves in a game, but may use much memory to do so. This thesis addresses the memory problem of B* by introducing two new variants called B^{*2} and df-B*. This chapter introduces the main topics discussed in this thesis and is structured as follows.

Section 1.1 introduces the topic of games as an environment for AI research and provides a relevant history of the field. Section 1.2 introduces game-tree search techniques and much of the search-related terminology used throughout this thesis. Section 1.3 introduces the specific category of game-tree search techniques we focus on in this thesis: game solvers, and provides an introductory explanation of B* and two other game-solving techniques. Section 1.4 provides the problem statement and research questions of this thesis. Finally, the overall outline of this thesis is provided in Section 1.5.

1.1 Artificial Intelligence and Games

Games have been at the forefront of Artificial Intelligence (AI) development for decades, and they continue to serve as a testing ground for new developments. Games like Chess, Settlers of Catan, Risk, Go, and many video games provide predictable environments on which AI agents can easily be trained, tested, and evaluated. Board games are simple to model, compared to real-world problems, with predefined rules and full knowledge of probability distributions, such as possible draws from a deck of cards or possible outcomes from a dice roll. In games, there is seldom a mismatch between the model the agent learns to play in and the environment, which is the game itself. Such that the only complexity is in designing the algorithm for playing the game itself. Video games may not always be easy to model, but the game itself is usually directly accessible, and its use does not bear many costs. So AI agents for video games can often be trained, tested, and evaluated directly in the game environment itself, which simplifies the development process and reduces costs. Finally, evaluation of agents, through the use of benchmarks, is generally simple in game environments. Single-player games and puzzles can simply be played to evaluate an agent's performance, whereas competitive games allow competitions with other AI agents or human players to be used as benchmarks.

Progress in game AI has been significant in the last 50 years. Whenever an AI agent defeats expert human players in a game for the first time, it marks an important milestone. Fairly cemented in the general consciousness is the computer players' supremacy in the game of Chess.

This may have started in 1997, when IBM's Chess program DeepBlue (Campbell, Hoane Jr, A. J., & Hsu, 2002) was able to beat the strongest Chess player at the time, Gary Kasparov, in 1997 using $\alpha\beta$ search and dedicated hardware. In Backgammon, master-level play was attained in 1994 by the reinforcement learning neural network agent TD-Gammon (Tesauro, 1994), which was developed throughout the 1990s. In 1998, TD-Gammon was defeated by the Backgammon world champion with a margin of 8 points over 100 games (Tesauro, 2010). In 1995, the parallel $\alpha\beta$-search driven Checkers program Chinook (Schaeffer et al., 1992; Schaeffer, Lake, Lu, & Bryant, 1996) halted competing against humans in tournaments and instead started work on a different task: solving the game of Checkers. The team behind Chinook succeeded in this in 2007 (Schaeffer et al., 2007), proving that perfect play of Checkers from the starting position leads to a draw, a draw that Chinook can guarantee through play against any possible counter strategy: the holy grail of game AI research. For a long time, it seemed that the game of Go was resistant to the conquest of AI techniques in games, so much so that when AlphaGo (Silver et al., 2016) defeated one of the strongest Go players in the world, Lee Sedol, by 4 games to 1 in 2016, it seemed to come out of nowhere for many. The Monte-Carlo Tree Search (MCTS) (Coulom, 2007) based program, guided by two large deep neural networks (DNNs), was initially trained on expert human players' games of Go. However, it achieved even better play as AlphaGo Zero (Silver et al., 2017) in 2017, which learnt entirely from self-play and had no initial knowledge of the game beyond its rules. The techniques developed for game-playing agents have advanced greatly throughout the decades, but they have also been applied to other fields beyond games, such as to chemical synthesis (Heifets & Jurisica, 2012; Franz, Mogk, Mrziglod, & Schewior, 2022).

1.2 Search and Games

In this thesis, specific techniques of game AI research will be discussed, called tree search techniques. The previously mentioned programs DeepBlue, Chinook, and AlphaGo are all tree search techniques, which may utilise other techniques as well, like machine learning for AlphaGo, which was the main kind of method used in TD-Gammon.

The 'tree' that is being searched in tree search techniques is called the game tree, which is a tree formed from the current and future possible states of the game. Looking at the game board, a game state is a frozen snapshot of all the information about the game that changes throughout the game and may affect the decisions players can make and the outcomes they have. Typically, we define a node in the game tree to be any such state where the 'current player' gets to make a decision, which determines the next state of the game. All possible decisions this player can make are their available moves, and all the nodes following each of these moves are the so-called children of that node. Nodes, or game states, are also often called game positions, which refers to the position of all the pieces on the board.

Eventually, throughout the play of the game, a position may be reached where the game is terminated, such as when it is won, lost, or tied. This is referred to as a terminal position, and the nodes representing these positions are called leaf nodes. Any other nodes are internal nodes: positions where the game is still going on. Leaf nodes may be given a value based on which player won the game, or based on what score each of the players received. This value may be relayed back to its parent node, the single node whose children include this one. For example, if all possible moves from the parent lead to a win for player 1, then this parent node has the same value as the leaf nodes at these win positions. This can be extended to assign a value to each internal node of the tree, based on all possible game states that follow from it. If we assume that all players play the game optimally, meaning they attempt to maximise their own score with full

knowledge of the game, then the value we assign to these nodes is called the game-theoretical value. In two-player zero-sum games where a win for one player determines a loss for the other, we can keep track of just one value at each node. One player tries to maximise this value, while the other player tries to minimise it. Hence, this is referred to as the minimax value of the sub-tree originating at that node.

If the true minimax values of the children of a node are known, you can essentially play the optimal next move at that node. However, the search tree of a game may often be much too large to search exhaustively, so intelligent techniques are needed to make the best possible estimate. This may include estimating the minimax value at internal nodes, known as internal node evaluation, and skipping the search of certain nodes, referred to as pruning. Many tree search techniques aim to estimate the minimax value of the current game position's node (the root node), like MCTS (Coulom, 2007) and $\alpha\beta$ search (Knuth & Moore, 1975). This can be achieved through simulated games originating at some position, as in the case of MCTS, where the average number of wins or losses in simulation determines the estimated value of a node. Nodes are also often evaluated using game-specific knowledge to assign some value to the game positions; the function that assigns these values is called a heuristic evaluation function. Heuristic evaluation functions may also be combined with MCTS, as is done for AlphaGo (Silver et al., 2016), where a neural network operates as a node evaluation function.

To determine the best move at the root, search is required: we need to traverse the tree by iteratively choosing which nodes to check next and incorporating what we learned into our final answer. This is what search is all about: what node to select next? When a node is selected, it is 'expanded', which means that we check for all the possible moves from that node and consider the resulting child nodes. Depth-first search selects one of the children of the current node to be expanded next, and then continues recursively, going deeper and deeper into the tree. Depth limits and the pruning of child nodes deemed 'irrelevant' for the current search ensure that the algorithm terminates on time. The benefit of depth-first search is that it is not constrained by the memory limits of computers. The performance of these algorithms can depend strongly on the order in which moves are considered, which is known as the move ordering. Another method is to search in a best-first manner, which often keeps the entire tree in memory, but selects the next node to be the 'best' node to expand next. This is typically a node chosen to optimise for the most 'progress' made per expansion, referred to as the most-informative, most-promising, or most-proving node. The benefit here may be a reduction in the total number of nodes searched. However, the memory limits may restrict the expanse of the search greatly compared to other methods. After selecting a node, its children are usually evaluated, and the information may be used to further advance the search and eventually determine the value of the children of the root node, which usually decides which move the agent plays.

However, the minimax value is not the only quantity that tree search techniques may aim to evaluate. The next section covers a special subclass of search techniques that do not aim to estimate values, but instead aim to prove specific quantities or qualities of game positions.

1.3 Adversarial Game Solvers

An important detail about value estimation techniques, as they are described above, is that they may not necessarily guarantee that the best move is selected. The process of estimating minimax values and pruning positions with unfeasibly high or low scores depends highly on the maximum depth searched and the 'correctness' of the evaluation function. Methods have been developed to counteract this, and one of them is the class of methods here called game solvers. If searched deep enough, specialised techniques can prove a win, draw, or loss at game

positions more effectively than value estimation, and they can prove other objectives as well. These techniques use evaluation functions, move orderings, and pruning procedures which are designed to provide a proof for whatever quantity is being tested while searching as few nodes as possible. For example, Proof-Number Search can be used to evaluate which moves could lead to a proven check in Chess, which is a useful game-specific quality of game positions in Chess. Additionally, game solvers are often used to solve puzzles, with or without an opponent. The following subsections cover three of these game-solving techniques: Proof-Number Search (1.3.1), λ-Search (1.3.2), and B* (1.3.3).

1.3.1 Proof-Number Search

Proof-Number Search (PNS) (Allis, van der Meulen, & van den Herik, 1994) is a technique used to prove binary statements about nodes in an adversarial game tree. This is typically applied to binary statements that can be determined from the sub-tree of a node or just the node itself for terminal nodes. So the statement "this node achieves a higher value than the alternatives" would not work for PNS, because it requires more information than what can be obtained from the sub-tree alone. Binary qualities like the existence of impasses, checks, wins, draws, or losses for the entire game or for a sub-problem of the game can be evaluated using PNS. A single PNS tree usually solves just a single binary question at a time, but multiple trees could be used to evaluate more complicated or categorical quantities.

PNS works by building a game tree, like other best-first techniques, containing proof-numbers and disproof-numbers for each node. Proof-numbers essentially count the minimum number of nodes that need to be evaluated as true (proven) in a node's sub-tree to evaluate whether its statement is true. Similarly, disproof-numbers count the minimum number of nodes in the sub-tree that need to be evaluated as false (disproven) to evaluate the node as false. At each step, the best node is selected to be expanded next, which is the 'most-proving node': the node with the adversarially decided highest chance to advance the proof. Search is terminated when the root node is proven or disproven.

PNS is effective and popular for solving games, as it was developed for this purpose (Allis et al., 1994). PNS and its variants are effectively used to help prove games or strengthen game-playing agents in various domains (Kishimoto, Winands, Müller, & Saito, 2012), including Shogi (Seo, Iida, & Uiterwijk, 2001), Othello (Nagai, 2002), Lines of Action (Winands, Uiterwijk, & van den Herik, 2004), Go (Kishimoto & Müller, 2005), and Connect6 (Xu, Ma, Tao, & Xu, 2009; Wu et al., 2011). The previously mentioned Checkers solver Chinook (Schaeffer et al., 2007) also uses depth-first Proof-Number Search (df-PN) (Nagai, 2002) when an $\alpha\beta$ search could not find a proven result in time. PNS and df-PN have also been applied to the fields of chemical (retro-)synthesis (Heifets & Jurisica, 2012; Franz et al., 2022).

1.3.2 λ-Search

λ-Search (Thomsen, 2000a) is a depth-first technique for proving the existence of a forced sequence of moves that achieves a tactical goal in adversarial zero-sum games. Unlike PNS (Allis et al., 1994), λ-search does not solve arbitrary binary goals. It is designed for goals that form a threat to the opponent, typically requiring a response, and provide a tactical advantage towards winning the game. A goal can be the win condition itself, such as a forced checkmate in Chess, or a tactical gain such as capturing a large territory in Go.

λ-Search iteratively builds higher hierarchies of so-called λ-trees. A λ^1-tree represents a forced threat sequence towards the goal, while a λ^2-tree represents a threat sequence towards a λ^1-tree, and so on for higher orders of λ^n-trees. This structure relies on the null-moves assumption: the

goal is guaranteed if the opponent passes their turn n times at a λ^n-node. Passing could also mean the opponent plays a move outside the λ^n-tree, as these trees contain only moves that belong to the relevant threat sequences. This can greatly reduce the branching factor of the search and makes the search more focused in games with many possible moves.

This technique may be applied to Chess or Go, but any game with threat sequences may work, as 4x4 tic-tac-toe was also provided as an example by Thomsen (2000b). λ-Search has since been used as a basis for other proof-search methods, such as RZOP (Wu & Lin, 2010) for Connect6. One difficulty noted by Thomsen is the possibility of inversions, where the opponent can turn a threat sequence into a counter-attack to escape the threat rather than defending against it. Thomsen outlined a possible direction for handling this issue, but the general problem remains open and has not been resolved in later work, such as RZOP.

1.3.3 B* Search

The technique of B* (Berliner, 1979) was developed to prove the optimal next move in games through the use of an evaluation function which provides a lower and upper bound on the true minimax value of the tree. Rather than proving a binary quality of the game state, B* maintains bounds on the minimax value for all nodes in its saved search tree. Updating these bounds eventually leads to the stop condition of B*: separation. Contrary to value estimation techniques, B* terminates consistently when the best move has been proven and no later, as that is when one move has a lower bound greater than any upper bound of the alternative moves. Value estimation guarantees selection of the best move only if the evaluation function is itself perfect, which would make the search unnecessary. B*, on the other hand, will always terminate with the best move if the provided bounds accurately contain the minimax value, which is a looser requirement than an exact match of the minimax value. In practice, however, the evaluation function might not be correct, in which case B* cannot guarantee the best move: the guarantees depend on the evaluation function, unlike Proof-Number Search, which is always correct for binary qualities of the game-state.

However, B* differs not only from value estimation in the use of bounds on the minimax value. It additionally employs two separate strategies for completing the proof. When the search is not confident about the currently most optimistic move, it expands its sub-tree further to improve its lower bound. When the confidence is increased, the program may choose one of two separate strategies: Prove-Best, or Disprove-Rest. Either we attempt to raise the lower bound further: Prove-Best; or we attempt to lower the upper bounds of the alternatives: Disprove-Rest. Either strategy may lead to the completion of the proof, but the strength of B* lies mainly in the conjunction of these two strategies. It is in a unique position as a game solver that utilises heuristics to estimate the game-theoretical value, which could skip major parts of the search that exact solvers may not be able to skip while remaining prudent.

Early research, like Palay (1982) and Berliner and McConnell (1996), presented promise for B* in the game of Chess. However, not much research was published after this, possibly due to the greater promise of depth-first value estimation methods. These can be pushed to further limits with specialised hardware, as shown by Campbell et al. (2002), which proved successful in the game of Chess. However, B* is a best-first search technique, which requires the game tree to be stored in memory. For small proofs, this is not a problem, and it does not hinder B* from performing well in certain domains, such as in the end-game play of Scrabble (Sheppard, 2002). But for much larger proofs, sub-trees may grow to sizes too large to store in memory, which currently cannot be solved with B* alone.

A recent report by Anema (2025) continues research on B* with the introduction of the Disprove-Best variant of B*, which improves the performance of B* on the specific sub-problem

of lowering the upper bound of 'false' best moves. Furthermore, the report also re-evaluates the performance of B* on large artificial game trees, which shows that the efficiency of B* grows further as the size of the problem increases. This is promising, as it shows that B* could perform well on large problems if memory allows it. As such, the report advises future research to focus on modifying B* to work on larger problems.

A similar task has been executed for the previously mentioned best-first Proof-Number Search (Allis et al., 1994). The PN^2 (Breuker, 1998) and df-PN (Nagai, 2002) algorithms expand Proof-Number Search to multi-level search and depth-first search, respectively, both of which allow PNS to search much larger problems. This thesis thus aims to apply a similar improvement to B*: expanding B* to work on larger trees.

1.4 Problem Statement and Research Questions

We have established that B* is a unique heuristic search technique that is also a game solver, with specialised strategies for proving the optimal move. One of its major drawbacks is the limit of memory usage due to its reliance on a stored game tree for best-first search. B* has been shown to work well on larger problems, so this problem is interesting to solve. The same has challenged the best-first game solver PNS before, which prompted research into depth-first and multi-level variants, some of which are still ongoing research. Therefore, the **problem statement** of this thesis is:

How can B be extended to search on much larger problems?*

To address this problem statement, we introduce two novel B* variants inspired by the PNS variants PN^2 (Breuker, 1998) and df-PN (Nagai, 2002). The multi-level variant B^{*2}, generalised to B^{*k}, reduces memory usage by keeping a smaller game tree in memory. The depth-first variant df-B* limits the memory footprint of B* to a fixed-size transposition table. This leads to the following three **research questions**:

1. *How can B* be adapted to multi-level (B^{*2}) or depth-first (df-B*) search?*
2. *How do B^{*2} and df-B* affect the memory usage of search compared to B*?*
3. *How do B^{*2} and df-B* compare to B* in terms of computational efficiency?*

1.5 Thesis Outline

This thesis is structured as follows. First, Chapter 2 gives an overview of adversarial game-solving techniques, including B*, Proof-Number Search, and their variants from existing research. Next, Chapter 3 introduces the novel multi-level B* variant B^{*2} and specifies how it has been implemented in this thesis. Chapter 3 also introduces the novel depth-first B* variant df-B*. This is followed by Chapter 4, which introduces the domain of artificial game trees and the methods for producing them as they are applied in this thesis. Experiments and their results are detailed in Chapter 5. Finally, our conclusions and recommendations for future research are provided in Chapter 6.

Chapter 2

Adversarial Game Solvers

This chapter describes the adversarial game-solving techniques B* (Berliner, 1979) and Proof-Number Search (PNS) (Allis et al., 1994), as well as some of their variants.

The chapter is structured as follows. First, Section 2.1 describes B* in greater detail and provides examples of strategy selection procedures for B*. Second, and finally, Section 2.2 describes the binary adversarial solver technique PNS, as well as the variant algorithms PN^2 (Breuker, 1998) and df-PN (Nagai, 2002).

2.1 B* Search

This section describes the best-first adversarial search technique B* (Berliner, 1979). The section is structured as follows. First, Subsection 2.1.1 describes B* by each of its major parts, which come together to form the search technique. Second, Subsection 2.1.2 describes several methods for selecting the strategy at the root during B* search.

2.1.1 Original Description

B* (Berliner, 1979) is a heuristic best-first tree-search algorithm which aims to prove the best next move in single-player (non-adversarial) or two-player (adversarial) games. This section describes B* for mini-max trees, which can model adversarial games, but setting each node to be maximising can model non-adversarial games as well.

Node Evaluation

B* uses a heuristic for the evaluation of internal nodes, as many other search techniques do. The heuristic provides a lower and an upper bound on the game-theoretical value for optimal play, which differs from value-estimation techniques: each node has two evaluations. Berliner (1979) refers to these evaluations as the optimistic and pessimistic values; instead, we use only the terms lower and upper bound to avoid confusion when switching between maximising and minimising nodes. The evaluation function itself will differ greatly across domains. Therefore, we assume that the evaluation function is correct when defining the algorithm and focus on creating a good heuristic at implementation.

Node evaluations are stored in the tree during search and get updated when more information is made available. Specifically, when a node is selected to be expanded, its children are added to the tree and evaluated according to the evaluation function. Then the node's bounds are updated

according to the update rule in Formula 2.1 for maximising nodes or Formula 2.2 for minimising nodes. This update is recursively propagated to the node's parent and its parents if it resulted in a change in bounds, until no bounds are altered. This is referred to as value back-propagation, after which the search continues from the last node whose bounds did not change, or the root if the backup reached the root node.

$$[Lower(x), Upper(x)] \rightarrow \left[\max_{c \in children(x)} Lower(c), \max_{c \in children(x)} Upper(c) \right] \tag{2.1}$$

$$[Lower(y), Upper(y)] \rightarrow \left[\min_{c \in children(y)} Lower(c), \min_{c \in children(y)} Upper(c) \right] \tag{2.2}$$

where x is a maximising node and y is a minimising node

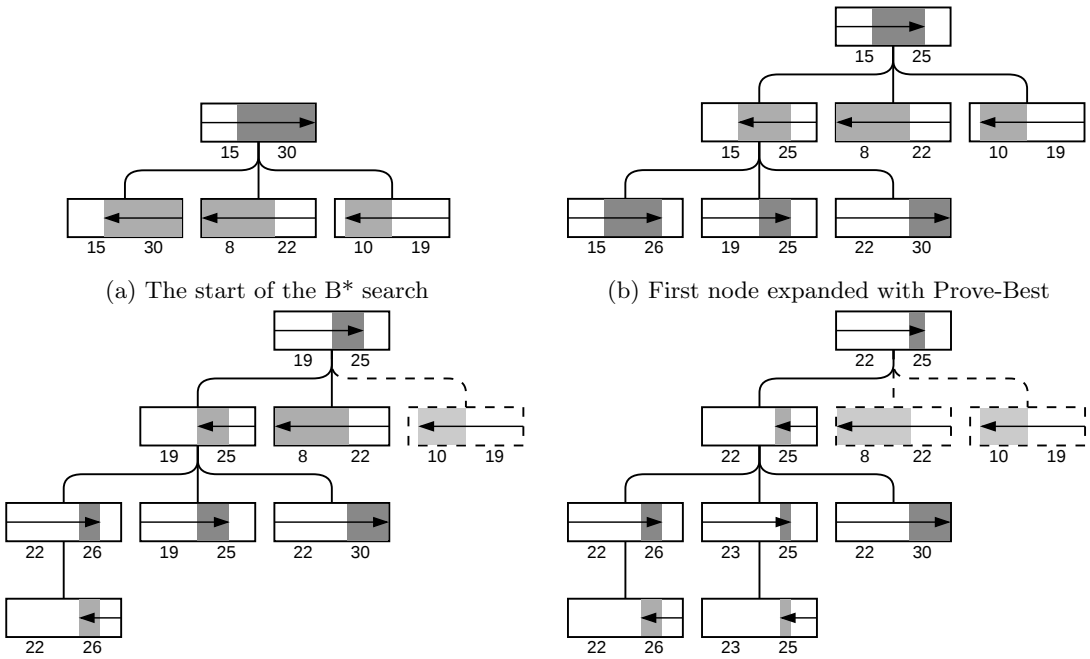
Stop Conditions

A best-first search using lower and upper bounds on the game-theoretical value has the advantage of knowing when we can stop the search completely, as described by Berliner (1979). Whenever the lower and upper bounds of a node are the same value, we know that further exploration of that node's sub-tree will not provide new information. However, B* can terminate the search much faster than this, due to the additional stop condition: separation. Separation occurs whenever a node's lower bound is greater than or equal to the upper bound of another. If, at the root node, the best node separates from all alternatives, then it has a game-theoretical value always greater than or equal to those alternatives and is therefore proven to be the best possible move. Including the separation of the best node from the second-best node as a stopping criterion for B* means it can terminate much faster while still having certainty that the search is complete.

Strategy Selection

When a value back-propagation alters the values of a node at depth 1, leading back to the root node, then we first select a strategy before selecting the next node. Strategy and node selection depend on whether the current node is maximising or minimising. As the root node is usually defined to be maximising, we describe selection from a maximising perspective without loss of generality: swap 'greatest' with 'lowest', 'raise' with 'lower', and 'upper bound' with 'lower bound' to obtain the minimising perspective. The strategy can be either Prove-Best or Disprove-Rest, and is an important part of B* search. Prove-Best is the strategy of selecting the ('best') node with the greatest upper bound at the root, such that its lower bound may be raised above the upper bounds of the alternatives, leading to separation. Disprove-Rest is the strategy of selecting the node with the second greatest upper bound at the root, such that its upper bound may be lowered below the lower bound of the 'best node', also leading to separation. The Disprove-Rest strategy is only applicable when the lower bound of the 'best node' is the greatest lower bound among all children at the root (Berliner, 1979). The possible benefit of choosing Disprove-Rest over Prove-Best is illustrated in Figures 2.1 and 2.2, which depict an example by Berliner (1979) where Disprove-Rest achieves a faster proof than Prove-Best.

Both strategies lead to the termination of the algorithm through separation, but the swiftest termination is reached through an intelligent combination of these two strategies. B* therefore employs a heuristic strategy selection protocol every time the search backs up to the root node. This choice of strategy is not trivial, as B* has been shown to work much better when both strategies are utilised. Some methods for strategy selection are discussed in Subsection 2.1.2.



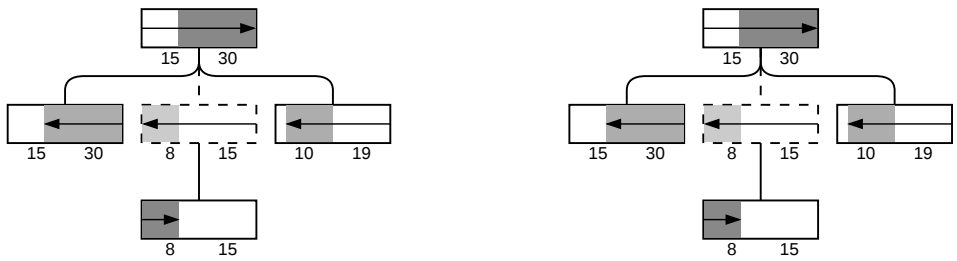
(a) The start of the B* search

(b) First node expanded with Prove-Best

(c) Second node expanded with Prove-Best

(d) Separation has been achieved with Prove-Best

Figure 2.1: An example tree, based on an example by Berliner (1979), showing the application of the Prove-Best strategy for B*. Each rectangle represents a node, with its lower bound (left) and upper bound (right) depicted underneath. Nodes with arrows pointing right are maximising, and nodes with arrows pointing left are minimising. Dotted lines denote nodes that have become irrelevant to the search.



(a) First node expanded with Disprove-Rest

(b) Separation has been achieved with Disprove-Rest

Figure 2.2: The same example tree, based on an example by Berliner (1979), from Figure 2.1 is depicted being explored with the Disprove-Rest strategy for B*. This example shows how the Disprove-Rest strategy can result in faster proofs than the Prove-Best strategy common for best-first search.

Node Selection

Finally, we describe how B* selects the next node to be expanded. As described above, B* selects either the node with the greatest upper bound or the node with the second greatest upper bound at the root node, depending on the decided strategy. However, the strategy may not always be selected. If the node with the greatest upper bound at the root does not also have the greatest lower bound, then Disprove-Rest can not lead to separation. Therefore, while the node with the greatest upper bound is not also the node with the greatest lower bound, B* uses the Prove-Best strategy. In addition, in best-first fashion, B* selects the best node at every depth other than the root for both Prove-Best and Disprove-Rest. This means to select the node with the greatest upper bound at maximising nodes and the node with the lowest lower bound at minimising nodes.

2.1.2 Strategy Selection

B*, as described above, requires a choice of strategy whenever the search backs up to the root. Berliner (1979) describes a heuristic approach to strategy selection, whereas Sheppard (2003) found a simpler approach to be effective in their implementation of B* for MAVEN in end-game play of Scrabble. This subsection describes some examples of strategy selection styles, although many others can exist and be applied to B*.

Naive Approaches

Naive approaches to strategy selection do not consider the structure of the tree or game-specific knowledge. The simplest example is to always select one strategy or the other. This results in **Always-PB** and **Prefer-DR** (Anema, 2025). Always-PB only ever selects Prove-Best. This is also referred to as 'Best-First' by Berliner (1979). Prefer-DR may select Prove-Best as well as Disprove-Rest, but always selects Disprove-Rest if it is applicable. As described before, B* may not select Disprove-Rest whenever the most optimistic node at the root is not also the least pessimistic node at the root, in which case Prove-Best is selected.

However, these variants do not take advantage of the benefits of the two strategies for B*. Naive approaches that may choose either strategy when both are applicable perform a lot better (Anema, 2025) while still being simple. One such method is the method employed by Sheppard (2003) for the MAVEN program in end-game play of Scrabble: **Alternated**. This simply alternates the chosen strategy between Prove-Best and Disprove-Rest whenever both are applicable. Following with Anema (2025), we implement this such that the alternation of strategies is halted when Disprove-Rest is not applicable and so that the first strategy selected is Prove-Best. Similarly, Anema also describes the **Randomised** approach, which randomly selects from either strategy when both are applicable. We can additionally define a parameter p which determines how often Prove-Best will be selected, which we take to be 50% unless stated otherwise.

Heuristic Approaches

Unlike the naive approaches described above, heuristic approaches usually incorporate information from the current state of the search tree to determine the next strategy. The approaches included in this thesis are two approaches described by Berliner (1979) in the original B* paper, but many others may be applied. The descriptions of these methods were modified to resolve ambiguities encountered during implementation. The x**D-criterium** selects Prove-Best or Disprove-Best based on the depth from which the upper bound of a node i has been obtained (d_i). If the sum $\sum_{i=2}^{x} {d_i}^2$ is greater than or equal to ${d_1}^2$, then Prove-Best is selected, otherwise

Disprove-Rest is selected. The children of the root node are sorted and numbered $1, 2, \ldots, x$ by decreasing upper bound, and any children with an upper bound below the greatest lower bound are excluded from the sum. The x**R-criterium** is the same as the xD-Criterium, but the values ${d_i}^2$, including $i = 1$, are divided by the range of i, defined as $Range(i) = Upper(i) - Lower(i)$.

2.2 Proof-Number Search

This section describes the best-first adversarial search proofing technique Proof-Number Search (PNS) (Allis et al., 1994) and two of its variants: PN^2 (Breuker, 1998) and df-PN (Nagai, 2002). The section is structured as follows. First, Subsection 2.2.1 describes PNS. Second, Subsection 2.2.2 describes PN^2. Finally, Subsection 2.2.3 describes df-PN.

2.2.1 Original Description

As described in the introduction, PNS (Allis et al., 1994) is a best-first algorithm for proving binary statements about nodes in an adversarial game tree.

Proof Numbers and Disproof Numbers. PNS represents the binary quality of a node with proof and disproof numbers. Proof and disproof numbers represent the minimum search effort required to prove or disprove each node, respectively. We often write these in brackets like so: (pn, dn), where the left 'pn' is the proof number, and the right 'dn' is the disproof number. Therefore, each non-terminal node is initialised with proof and disproof numbers $(1, 1)$, as at least one expansion is needed to either prove or disprove any leaf node. When a node is proven, we represent this with a proof number of 0 paired with a disproof number of ∞ like so: $(0, \infty)$. Likewise, disproven nodes are represented by $(\infty, 0)$. PNS by default keeps searching until the root node is either proven $(0, \infty)$ or disproven $(\infty, 0)$.

AND/OR Trees. For PNS specifically, it is useful to see the adversarial game tree as an AND/OR tree. This can be seen as OR nodes for the current player, and AND nodes for the opponent. The first node is assumed, without loss of generality, to be an OR node, and alternating depths consist of either AND or OR nodes. For any node n, if n is an OR node, then proving any child of n proves n. However, n is only disproven when all of its children are disproven. If n is an AND node, then disproving any child of n disproves n, but n is only proven when all of its children are proven. This distinction between AND and OR nodes determines how (dis)proof numbers are updated at each node, as well as how the next node is selected at each node.

Most-Proving Node (MPN). PNS depends on the core idea of the Most-Proving Node (MPN). The MPN is a leaf node of the tree, which, if proven or disproven, contributes directly to the progress of the search at the root. The MPN is determined by choosing the node with the minimum proof number at OR nodes, and choosing the node with the minimum disproof number at AND nodes, until a leaf node is chosen.

Updating Values. In PNS, when a node n is expanded, or one of its children is altered, then the (dis)proof numbers of n will be determined as follows. If n is an OR node, then the proof number of n is the minimum proof number among the children of n, and the disproof number of n is the sum of the disproof numbers of all children of n. If n is an AND node, then its disproof number is the minimum disproof number among its children, and the proof number is the sum of proof numbers of all its children.

PNS Algorithm. The PNS algorithm (Allis et al., 1994), unmodified, is quite simple as a whole. As mentioned before, PNS keeps searching until the root node is either proven or disproven, iterating the following steps until that is the case:

1. **Select** the most-proving node (MPN).
2. **Expand** the MPN by adding all of its direct successors to the tree as new nodes.
3. **Evaluate** each of the newly added nodes as either *proven* $(0, \infty)$, *disproven* $(\infty, 0)$, or *unknown* $(1, 1)$.
4. **Backpropagate** along the path to the root, updating all proof and disproof numbers along the way, until they no longer change.

This basic implementation of PNS only depends on the structure of the tree and does not require any game-specific knowledge besides what is needed to generate and evaluate nodes as either proven, disproven, or unknown.

2.2.2 PN^2: Search on Larger Trees

PN^2, as described by Breuker (1998), is a multi-level PNS variant with reduced memory usage. PN^2 effectively replaces the '**evaluate**' step of PNS, as described above, with another PNS search. This has the effect of increasing the information gained from each expansion of the first-level PNS, which reduces the overall size of its tree. However, to achieve this, the majority of the search effort is discarded at the end of every second-level PNS search, which negatively affects the computational cost of the search. To control this effect, a size limit is applied to the second-level search, which is terminated early when the second-level tree exceeds its size limit. The second-level tree-size limit thus determines the trade-off between memory usage and computational efficiency.

Breuker (1998) describes two main approaches for setting this tree-size limit. The first of which is to set the limit of the second-level search to be the size of the first-level tree. We will refer to this approach as **max**. The second approach is to take a fraction between 0 and 1 of the previous limit. Breuker (1998) proposes the use of a logistic growth function to determine this fraction. Let x be the size of the first-level tree, then $f(x)$ is the fraction of x to use as the second-level tree-size limit, as follows:

$$f(x) = \frac{1}{1 + e^{\frac{a-x}{b}}} \tag{2.3}$$

where, a and b are the logistic growth parameters. We will refer to this approach as **logistic**. With this, the actual tree-size limit of the second-level PNS search is the minimum of $x \cdot f(x)$, or x for **max**, and the remaining available memory $N - x$, where N is the total available memory.

2.2.3 df-PN: Search on any Size Tree

In the development of PNS techniques, a technique that helped reduce, or even trivialise, the memory usage of PNS beyond PN^2 (Breuker, 1998) is df-PN (Nagai, 2002). Unlike PN^2, df-PN does not rely on storing the search tree itself in memory. It instead uses only the execution stack and a transposition table (TT) to store progress of the search, similar to depth-first search techniques. Despite the name, df-PN behaves mostly like a best-first algorithm. It selects nodes in much the same way PNS does, but does not back-up values to the root as frequently as PNS. This is thanks to the use of value thresholds on the (dis)proof numbers, which control when the search gets backed up. The overall algorithm is structured as a multiple iterative deepening search, which repeatedly calls itself on child nodes until a (dis)proof number threshold is met. The thresholds of a node are chosen at each step based on the (dis)proof numbers of its alternatives.

Chapter 3

Saving Memory for B*

The B* (Berliner, 1979) technique, as described in the previous chapter, has been extended to PB* (Palay, 1983) and Disprove-Best B* (Anema, 2025) to increase its efficiency as measured by the number of nodes expanded to solve optimality for an adversarial game-tree. Whereas Palay and Berliner and McConnell (1996) saw the most promise in a probabilistic approach for further development of B*, Anema notes promise in extending B* to a form with reduced memory usage. This chapter explores this by describing ways to reduce memory usage in B* and by describing two B* variants with reduced memory usage.

This chapter is structured as follows. First, Section 3.1 describes some adjustments we made to B* to reduce its memory usage and improve its performance on the test domain. Second, Section 3.2 introduces B^{*2}, a multi-level B* technique inspired by PN^2 (Breuker, 1998), which reduces the memory usage of B* at the cost of additional computation. Finally, Section 3.3 introduces df-B*, a multiple iterative deepening technique inspired by df-PN (Nagai, 2002), to reduce the memory usage of B* even further with a transposition table and value thresholds.

3.1 Additional B* Adjustments

This section describes additional adjustments to B* to improve the memory usage and general performance on the test domain of artificial game trees. However, these adjustments are not domain-specific and may be applied to any domain.

Memory Management. The main adaptation to reduce the memory usage of B* is also described by Berliner (1979): **pruning irrelevant sub-trees**. Any child node, anywhere in the tree, with an upper bound below its parent's lower bound or a lower bound above its parent's upper bound can not affect the outcome of the search. Specifically, if a node's bounds do not overlap with its parent, and its removal does not lead to a change in the bounds of its parent, we prune the node and its subtree from memory entirely. The pruning of nodes generally occurs at the value update of nodes during back-propagation.

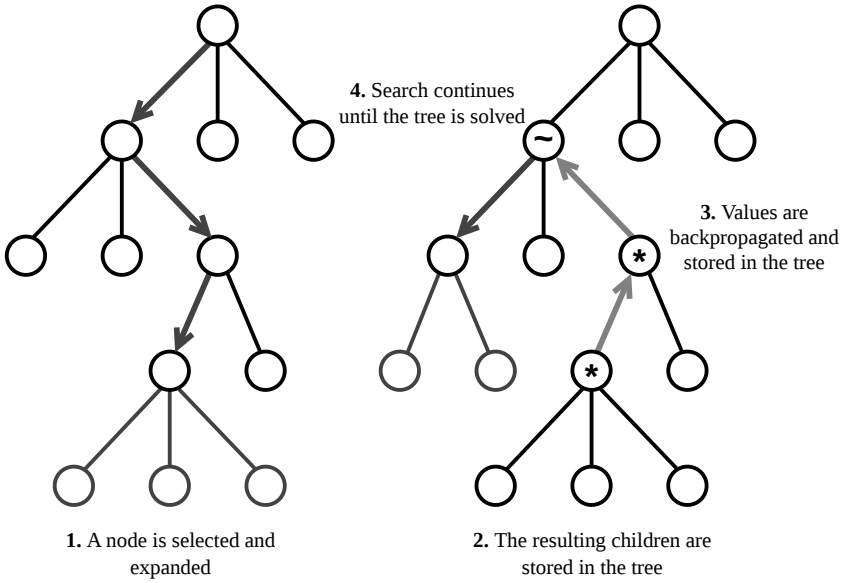


Figure 3.1: This diagram depicts how memory is handled in B* search. The entire tree is stored in memory, and expansions and evaluations are performed as described by Berliner (1979). Nodes are only removed from the tree if they have become irrelevant to the search through separation or through the separation of an ancestor.

Novel Modifications. We additionally describe two modifications to B* that preliminary experiments showed make B* perform better and make B* more comparable to B*^{2} (see Section 3.2) on our choice of test domain as described in Chapter 4.

We introduce a **maximum to the number of consecutive back-propagations** without returning to the root node. A counter is incremented by 1 each time the search back-propagates and continues the search from an internal node. If this node is the root node, then the counter is reset to 0. The search is forcibly backed up to the root whenever this counter exceeds the maximum number of rootless updates M. In this thesis, we always use $M = 1000$. This modification is included due to the infinite nature of the domain (Chapter 4) used in this thesis.

We also introduce a **modified selection protocol** at internal nodes (see Algorithm 1). At any node, not including the root node, the selection of the next node defaults to the most optimistic child node N_{opt}. However, if N_{opt} is not equal to the least pessimistic node N_{pes}, then we may select N_{pes} instead. We may only select N_{pes} as the next node if its range is greater than 0, such that N_{pes} is not a terminal node and has not yet been solved. If these conditions are met, we select N_{pes} with a probability of P_{alt}. Preliminary experiments showed that a choice of $P_{alt} = 0.3$ gives the best performance on our domain. In this thesis, we therefore always use $P_{alt} = 0.3$. We decided to include this modification to adjust for a measure of uncharacteristically low performance of B* on large artificial game trees compared to B*^{2} in preliminary experiments.

Algorithm 1 selectAlternative(leastPes, mostOpt, P_{alt}) $\rightarrow$ nextNode

```
if leastPes is not mostOpt and leastPes.range > 0 and randomly(P_alt) then
    nextNode := leastPes
else
    nextNode := mostOpt
```

3.2 A Multi-Level Approach to B*

Several approaches have been applied to reduce the memory usage of the best-first adversarial solver technique PNS by Allis et al. (1994). These approaches can serve as inspiration for B* (Berliner, 1979), as it is also a best-first adversarial solver. One such approach is PN^2: a two-level PNS engine with the promise of "PN Search with small memory" (Breuker, 1998). This section describes a B* variant inspired by PN^2 to allow B* to search larger trees with the same memory: The B^{*2} algorithm.

This section is structured as follows. First, Subsection 3.2.1 describes the B^{*2} algorithm conceptually, stating its components and caveats. Second, Subsection 3.2.2 discusses the implementation detail of second-level tree-size limits and lists some example functions to determine the size of the second-level tree based on the first-level tree. Finally, Subsection 3.2.3 describes an extended variant of B^{*2} that supports more than two levels of the search tree, generally: B^{*k}.

3.2.1 B^{*2} Search

The main concept of B^{*2} is the same as that of PN^2 (Breuker, 1998): a two-level search provides more information per node evaluation, leading to earlier termination and a smaller tree to store in memory. B^{*2} is a search consisting of two levels of B* search. The first level builds the tree in the same manner as B* (Berliner, 1979), as it is described in Section 2.1, but does not use the usual evaluation function for the evaluation of the next node. Instead, a second-level search is initiated with the node to evaluate as its root. This second-level search may have a different strategy selection procedure and different stop conditions as the first-level search. When the second-level search terminates, the values backed up to its root are used to update the first-level search, and the rest of the second-level tree is discarded. This second-level search thus functions as a more 'informative' evaluation function for the first-level B* search tree, trading additional computation for lower memory usage.

Immediate Evaluation. B^{*2} is inspired by PN^2 (Breuker, 1998), but it differs in some aspects due to the inherent differences between PNS and B*. An important difference is as follows. PN^2 employs delayed evaluation for the first-level search and immediate evaluation for the second-level search. The concept of delayed evaluation is mentioned by Allis et al. (1994) for PNS and refers to waiting to evaluate a node until it has been selected. Immediate evaluation instead refers to evaluating a node immediately after it has been added to the tree through expansion of its parent node. Delayed evaluation is relevant for reducing computation when the evaluation function is heavy to compute, such as for PN^2. However, we recommend the use of immediate evaluation for both the first- and second-level search of B^{*2}. Delayed evaluation for B* has limited utility and has not yet been formalised. If a maximising node has one or more of its children not yet evaluated, its lower bound will be a correct lower bound for the game-theoretical value, but its upper bound may not be, as it may widen when the last children are evaluated. This complicates the implementation of delayed evaluation in B*, as node selection and pruning depend on the correctness of bounds at all stages of the algorithm. Delayed evaluation for B* is not completely ruled out as a possibility, but has not been implemented in this research for this reason. Therefore, we apply only immediate evaluation for all levels of the search.

Stop Conditions. The second-level search for B^{*2} may have different stop conditions than the first-level search, which are checked when the search backs up to its root. As usual with B*, the first-level search stops when separation is reached for the best child node of the root. Additionally, the search may terminate when physical limits on some resource or a time limit are

reached, such as when the process runs out of memory. The second-level search may terminate earlier as its goal is not to prove the optimality of some move at its root, but to provide more information for the first-level search. We describe four stop conditions for the second-level B^{*2} search: (1) separation, (2) size limit, (3) shallow irrelevance, and (4) deep irrelevance.

The first stop condition, **separation**, is the main stop condition for B* search, which is to halt the search when the proof for the best move at the root is complete. For the second-level search, this amounts to stopping the search when only one of its children is relevant to the search. This stop condition is beneficial to B^{*2} for two reasons: (a) separation reduces the set of children of a node, leading to fewer nodes being stored in the first-level tree when this node is expanded later, and (b) if continued after separation, the second-level search can no longer choose between Prove-Best and Disprove-Rest, which reduces the effectiveness of B* at that level.

The second stop condition is the **spatial limit** imposed on the search. This is triggered when the total number of nodes stored in the second-level tree exceeds a specified limit. How this limit is determined is described in Subsection 3.2.2, which regulates the degree to which it is enforced. To extend the search before reaching these spatial limits we implement the pruning of irrelevant sub-trees at the second-level search tree, just as we do for the first-level search.

The third stop condition is that of **shallow irrelevance**. A node is irrelevant when its bounds are always equal to or worse than at least one of its sibling nodes. If the second-level root is irrelevant with respect to the first-level tree, we say that the current sub-tree has become irrelevant. Shallow irrelevance stops the second-level search as soon as the direct irrelevance of the second-level root is detected.

The fourth and final stop condition is **deep irrelevance**. This is similar to shallow irrelevance, but in addition to checking the irrelevance of the current sub-tree at the parent of the second-level root, we check it along the entire line of ancestors leading up to the first-level root. This check continues until no node values are updated, just like a normal backup in B* search (Berliner, 1979). Additionally, if the backup leads back to the first-level root, we stop the second-level search if separation of any kind has been reached at the first-level root, as this completes the first-level search as well. Deep irrelevance checking adds a decent amount of extra overhead, as it requires a complete backup of values to the first-level root every time the second-level search reaches its root. This is why we present it separately from shallow irrelevance.

Strategy Selection. Strategy selection is performed whenever B* backs up changed bounds to the root. For B^{*2}, strategy selection may differ from the first-level search to the second-level search. The first-level search may use more computational effort to determine its strategy, as it can be offset by reducing the number of second-level searches that need to be made. Whereas the second-level search will perform strategy selection many more times during the search, and it is thus preferably fast. Therefore, it is advised to use heuristic approaches for strategy selection of the first-level search and to use a fast strategy selection for the second-level search. This can be, for example, a depth-based heuristic as described by Berliner (1979) for the first-level search and a naive strategy selection (see Subsection 2.1.2) for the second-level search.

Preservation of Bounds. Finally, we discuss additional nuance on how B^{*2} may keep or discard part of the second-level search tree after it concludes, as determined by the stop conditions described above. Because PN^2 uses delayed evaluation, Breuker (1998) decided to keep the children of the second-level root in memory after the second-level search terminates, unless the root was solved. However, Breuker does not mention whether the computed (dis)proof numbers of these children were also preserved, or how they were incorporated into the following first- and second-level searches. We therefore describe four separate approaches to bounds preservation in multi-level search, for the context of B^{*2}:

1. **Full disposal** is the absence of bounds preservation. We keep the updated values of the second-level root and store them directly in the first-level tree. The rest of the second-level tree is discarded and will need to be re-created upon further examination of this node. This is the approach that PN^2 usually does not use, but that we have implemented for B^{*2} as it is clear, consistent, easy to implement, and works well.

2. **Re-search** is the approach most likely used by Breuker (1998), where the children of the second-level root are not discarded after the second-level search terminates, unless this root is solved. As a result, the next possible nodes do not need to be recomputed if the second-level root node is expanded again later. The saved children may also retain their bounds as they were computed in the second-level search. These bounds can provide additional information to move ordering, node selection, and second-level stop conditions, such as shallow irrelevance, as described above. However, these children are not kept in the first-level tree as computed nodes and will be re-expanded upon further evaluation.

3. **Windowed** is an additional consideration for the re-search approach. Aside from providing information to move ordering, node selection, and stop conditions, the saved bounds can also function as a search window. For B*, this works by pruning encountered nodes immediately if they appear irrelevant to the search when compared to the search window, while they may still appear to be locally relevant.

4. **Keep bounds** is the approach where the children of the second-level root are not discarded, and are instead directly stored in the first-level tree. Children for which initial bounds are already present are then not evaluated further with a second-level search. This way, second-level searches are only initiated on alternating depths. These saved bounds are less informative than the root bounds resulting from the second-level search, but do not require an additional search as with the other methods.

One key difference between B^{*2} and PN^2 is that B^{*2} does not perform delayed evaluation. As a result, children kept after the second-level search terminates are not necessarily used immediately afterwards. On the contrary, B* expands a node n, which initialises a second-level search on each child c of n, then re-evaluates the bounds of n. A likely result is that the bounds of n are altered, which results in a back-propagation of the search. Therefore, without delayed evaluation, we recommend the **full disposal** approach to bounds preservation for B^{*2}.

3.2.2 Second-Level Tree-Size Limit

Similar to PN^2 (Nagai, 2002), we apply a tree-size limit to the second-level search of B^{*2}. Subsection 2.2.2 describes two approaches for determining tree-size limits in PN^2. These approaches are also applied to B^{*2}, as described here.

The **max** tree-size limit is the technique of setting the size limit of the second-level search equal to the current size of the first-level tree. This is a simple and effective way to set the second-level limit. However, Breuker (1998) notes how this approach made PN^2 take much longer to solve small problems. To address this, the **logistic** size limit was introduced. Here we take the limit of the second-level tree to be a fraction $f(x)$ of the size of the first-level tree x, which is defined as:

$$f(x) = \frac{1}{1 + e^{\frac{a-x}{b}}} \tag{3.1}$$

where a and b are the logistic growth parameters, which control for the shape of the logistic growth curve. For our experiments, we applied values of $a = 500{,}000$ and $b = 50{,}000$.

Ultimately, for both approaches, we restrict the actual tree-size limit by the remaining available memory $N - x$, where N is the total available memory. Such that the memory limit for **max** is:

$$\text{MaxLimit} = \min\left[x, N - x\right] \tag{3.2}$$

And the memory limit for **logistic** is:

$$\text{LogisticLimit} = \min\left[\frac{x}{1 + e^{\frac{a-x}{b}}}, N - x\right] \tag{3.3}$$

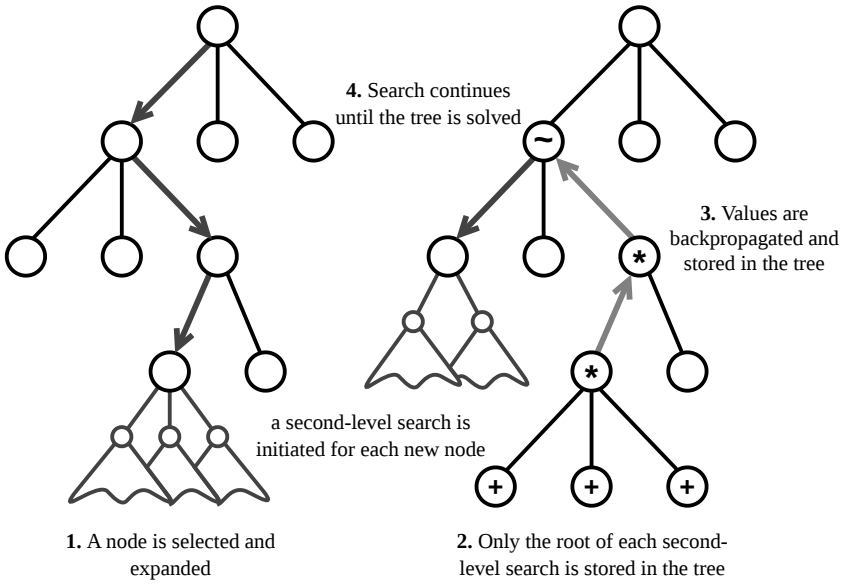


Figure 3.2: This diagram depicts how memory is handled in B*^{2} search. Only the first-level tree is stored in memory throughout the search. Evaluations are performed through a second-level B* search, the tree of which is discarded after its termination. The values of newly expanded nodes are more informative as a result, leading to a smaller tree overall.

3.2.3 Extension to Many-Levelled Search

The B*^{2} search technique described above allows B* to search larger trees, which would usually not fit in memory, by structuring the search in two levels. This subsection describes the generalisation from two-level B*^{2} to k-level B*^{k} search. This technique reduces memory usage further, at the cost of additional computation, for increasing values of k.

Motivation

The idea of utilising a second-level search as an evaluation function for the first-level search, which inspired PN^2 (Breuker, 1998), was taken from an earlier paper on Bayesian Search by Baum and Smith (1997). The paper mentions multi-level search as a possible extension to reduce memory usage to "roughly a square root", while "imposing speed reductions of only a constant factor". This is also what allows B*^{2} to reduce its memory usage at the cost of additional computation. Effectively, the multi-level search enhances the evaluation function for B* search, which allows

the search to terminate earlier. Similarly, we can use a B* search as the evaluation function of a B^{*2} search to create a three-level B^{*3} search.

When we apply the prediction of square-root memory usage by Baum and Smith (1997) to B*, then a k-level search may expect roughly $\sqrt[k]{N}$ of nodes stored in memory compared to N nodes for a plain B* search. Therefore, the motivation behind B^{*k} is that, if a problem is too large to be solved with B* or B^{*2} within the memory limitations, then some greater choice of k may exist for which B^{*k} can solve the problem. However, in all cases, there has to be at least enough memory available to store the execution stack of the search, which is in the order of the maximum depth reached by the search. Additionally, the square-root prediction, combined with the overhead of search, suggests diminishing returns for larger values of k. However, this does not matter as much if B^{*k} allows problems to be solved that could not be solved before.

Overhead

When implementing and applying B^{*k}, note that the benefit of increasing the number of levels k is primarily in the reduction of memory usage. Every additional level of nesting increases not only the number of nodes expanded by the search, but also the level of overhead for each step of the first-level search. Increasingly nested levels of search contribute more to computational overhead while contributing less, per tree, to the total search effort. Any overhead from search initialisation, stop conditions, strategy selection, and size limit calculations is magnified for the highest levels of the search. Therefore, it is good to minimise k for the resources available, to increase the efficiency of the algorithm.

Strategy Selection

Different strategy selection procedures may be chosen for every level of the B^{*k} search. More complex methods, like the depth-based criterion by Berliner (1979), require additional overhead, which is magnified at higher levels of the search. At these levels, we find that it is a safe default to always select Prove-Best. Alternating between Prove-Best and Disprove-Rest, or randomly selecting the strategy, are also good choices for higher-level searches. The latter can be tuned by choosing the chance of selecting Prove-Best p (see Subsection 2.1.2) for the best results. Similarly to B^{*2}, it may be worthwhile to expend extra effort for strategy selection in the first-level search, as each expansion of the first-level tree initiates a large amount of search effort by a B^{*k-1} search on the children of the expanded node.

Tree-Size Limits

Tree-size limits get more complex with multiple levels of search, as there are many possible configurations. However, it may be ill-advised to maximise all limits. If all levels of the search use the **max** tree-size limit, then each k-level search creates and deletes as many nodes as are present in the first-level tree. This would maximise the amount of search that is discarded after a nested search terminates, whereas a smaller search might have saved on computation. However, stricter tree-size limits do increase memory usage, but this effect is less noticeable for highly nested searches. It is therefore advised to taper off the tree-size limits of higher-level searches, such that the second-level search has more allocated memory, whereas the third-, fourth-, and higher-level searches get decreasing amounts of memory. For example, we found that a configuration for B^{*3} with a **logistic** size limit, as described in Subsection 3.2.2, for the second-level tree and a small fixed size limit for the third-level tree showed promise.

Stop Conditions

B^{*k} may also have different stop conditions for each level of the search. The second-level stop conditions can be chosen the same way as for B^{*2}. Higher-level searches magnify the overhead of all operations, including the stop condition, which is checked whenever the search backs up to the root. Separation is generally a cheap condition to check, so it can usually be included at all levels of the search. Shallow and deep irrelevance, as described in Subsection 3.2.1, have some additional overhead, which may outweigh their benefits for higher-level searches. Additionally, if tree-size limits are set to taper off for higher-level searches, then stop conditions likely do not affect the higher levels of the search as much. It is more likely that the search is terminated by running out of resources when the tree-size limits are small. Reducing the complexity of stop conditions is therefore advised for higher levels of the search.

3.3 A Depth-First Approach to B*

Another approach to reduce the memory usage of best-first search techniques is to restructure the search as a depth-first search. An example of this is the PNS variant df-PN (Nagai, 2002), which we describe in Subsection 2.2.3. We apply this approach to B* as well, which results in the depth-first B* variant df-B*. This section describes this novel search technique and is structured as follows. First, Subsection 3.3.1 describes the df-B* technique from its inspirations to its formal definition, and finally, its pseudocode. Second, Subsection 3.3.2 presents some challenges and implementation details for df-B* that may need to be taken in consideration when applying this technique to other domains.

3.3.1 df-B* Search

For the depth-first approach to B*, a major difference is the lack of a search tree stored in memory, as B* and B*[2] do. Instead, depth-first search relies on a transposition table (TT) to store the results of sub-tree searches. The technique we introduce, df-B*, is heavily inspired by df-PN (Nagai, 2002), which we describe in Subsection 2.2.3. As with df-PN, we use thresholds to control the effort expended by recursive and iterative calls of the search function. However, unlike df-PN's (dis)proof numbers, B*'s bounds and their subsequent thresholds do not provide true information on the difficulty or expanse of search needed to prove a certain goal. They may indicate it, but there are no guarantees beyond those that ensure B* terminates exactly when the optimal move is proven. This implementation may therefore come with some caveats resulting from that key difference. First we describe the adaptations needed to construct df-B*, as viewed from the perspecfive of df-PN.

From df-PN to df-B*

The main additions and modifications needed to turn df-PN into df-B* come from the differences between PNS and B*.

First, PNS uses the same search procedure at the root as anywhere else in the tree, but B* does not, and df-B* cannot either. For B*, node selection at the root differs from node selection at all other nodes. We therefore describe two main loops of the df-B* algorithm: root-level and low-level. The root-level loop, or simply root-level search, repeatedly decides the strategy and calls low-level searches until separation has been reached. The low-level search iteratively calls itself through Multiple Iterative Deepening (MID), controlled by node value thresholds, while utilising a TT for storage, similar to df-PN (Nagai, 2002).

Second, as mentioned earlier, (dis)proof numbers provide different information than B* minimax bounds do. Nagai (2002) describes df-PN with thresholds propagating based on (dis)proof numbers of competitors, either by summing them or by taking the value of a competitor +1. Bounds for B* are heuristic values and thus are not easily described as discrete units of effort. To address this, we take inspiration from the $1 + \epsilon$ trick for df-PN by Pawlewicz and Lew (2007): thresholds grow by a constant factor multiplied by the (dis)proof numbers. Instead of setting the 'target' of a search call to the necessary threshold +1, we let it grow by a multiplicative factor of $1 + \epsilon$. This extends well to B*, as we can take the threshold T, which, when reached, would change the behaviour of B* but not terminate the search, and set our new threshold to $T \pm \epsilon \cdot (U - L)$. This way of setting thresholds reduces frequent switching between two or more nodes with similar values, by taking longer to stop searching a branch that is no longer optimal. To make df-B* behave as closely to B* as possible, we can set ϵ close to 0. The exact value of ϵ may depend on the domain, but is always a factor of the bounds of a node. If two evaluation functions can be mapped linearly, then df-B*(ϵ) will behave the same way on both.

Third, PNS only knows one strategy and one type of behaviour, but B* has two strategies: Prove-Best and Disprove-Rest. Strategies for df-B* are handled in the root-level search and are determined heuristically. Heuristics that can be used for B* should also apply to df-B*, given that the necessary information does not require storing the search tree in memory. An adaptation specifically for df-B* is the addition of the parameter σ , which performs the same role as ϵ, but only for the root-level search. Increasing ϵ makes the search behave more aggressively and more efficiently in space-limited scenarios (Pawlewicz & Lew, 2007), at the low-level search. But σ determines how often the search will back up to the root, at the root-level search. In df-PN, there is no crucial difference between ϵ at the root and ϵ at internal nodes. But for df-B*, the amount of leniency in thresholds at the root controls the frequency of strategy reconsideration, the quality of which affects the efficiency of B* (Berliner, 1979). The parameters ϵ and σ thus control the balance between memory efficiency and the frequency of B* strategy selection during the search.

An Overview of df-B*

We implement df-B* based on an unmodified B* search, where the only adaptations made are those required to structure B* as a depth-first search as inspired by df-PN. The following sketch shows an overview of df-B* divided into the root-level and low-level search procedures:

Root-level df-B* : $\sigma \in \left(0, \frac{1}{2}\right)$
At root node r, sort children $N_1, N_2, \ldots, N_b$ by decreasing upper bound;
Define upper and lower bounds as U_i and L_i for any node N_i;
While $L_1 < U_2$ ***do:***
  If $L_1 < \max L_i$: Set $s = 1$;
  Else heuristically decide $s = 1$ (PB) or $s = 2$ (DR);
  Start searching at N_s until thresholds exceeded:
    $t_s = \min\left[U_2, L_s + \sigma \cdot (U_s - L_s)\right]$;
    $\tau_s = \max\left[L, U_s - \sigma \cdot (U_s - L_s)\right]$;

Low-level df-B* : $\epsilon \in (0, 1)$
Searching at node n with bounds L, U and thresholds t, τ;
While $L < t$ **and** $\tau < U$ **and** $L < U$ ***do:***
  If n is Minimising:
    Sort children $N_1, N_2, \ldots, N_b$ by increasing lower bound;
    Start searching at N_1 until thresholds exceeded:
      $t_1 = \min\left[t, L_2 + \epsilon \cdot (U_1 - L_1)\right]$;
      $\tau_1 = \tau$;
    Maintain $L = \min L_i$ and $U = \min U_i$;
  If n is Maximising:
    Sort children $N_1, N_2, \ldots, N_b$ by decreasing upper bound;
    Start searching at N_1 until thresholds exceeded:
      $t_1 = t$;
      $\tau_1 = \max\left[\tau, U_2 - \epsilon \cdot (U_1 - L_1)\right]$;
    Maintain $L = \max L_i$ and $U = \max U_i$;

Thresholds at the root- and low-level search are chosen to stop the search immediately when separation is reached at the root, so there is no extra computation regardless of ϵ or σ. We have also provided pseudocode for the root-level df-B* (see Algorithm 2) and the low-level df-B* (see

Algorithm 3). The pseudocode shows the usage of a transposition table (TT) for df-B*, as this technique is designed to work with a TT. The main information stored in the TT for df-B* is the lower and upper bounds L, U for each node n. Thresholds do not need be stored, as they are computed from the top down during the search. If a TT entry for n already exists, then we keep the values that give the tightest bounds (by smallest range $R = U - L$). If a TT entry exists for a different node m with the same key as n, then we require a replacement scheme (Nagai, 2002). For this thesis, we apply the 'always replace' scheme, which means we always overwrite the entry of m in favor of the newer entry n. Other replacement schemes are discussed in Subsection 3.3.2, along with other information which may help with the implementation of df-B*.

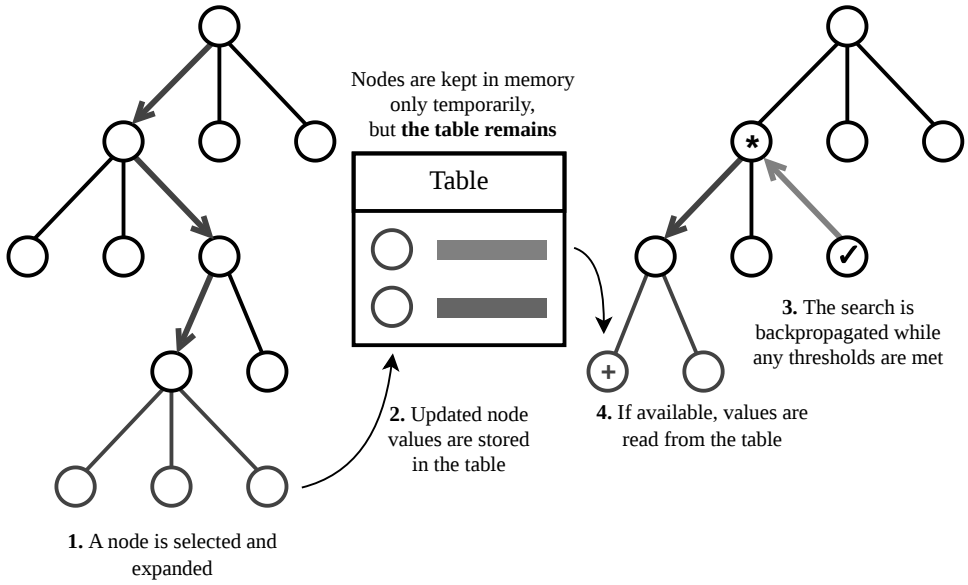


Figure 3.3: This diagram depicts how memory is handled in df-B* search. Only the direct line of ancestors of the current node and their children are stored in memory during the search. Nodes are constantly getting discarded, so their values are stored in a transposition table as the primary information storage of the algorithm. To reduce the loss of computational effort, the search back-propagates less frequently than B* or B*^{2} do.

3.3.2 Challenges

For this thesis, df-B* has been implemented as its minimal viable version as tested on the domain of artificial game trees. This subsection discusses some difficulties and challenges that may be encountered when implementing df-B* or when applying df-B* other domains.

Heuristics. Unlike df-PN, df-B* requires an evaluation function to work, as well as a, possibly heuristic, strategy selection procedure. Both may need to be adjusted for specific domains. The parameters ϵ and σ may also need to be adjusted to work together with the evaluation function as well as with the strategy selection procedure. Preliminary experiments showed that $\sigma = 0.1$ and $\epsilon = 0.3$ worked best on the domain of artificial game trees.

Algorithm 2 df-B*(r, σ, ϵ, H)

```
N_i := CreateChildren(r)
L_i, U_i := GetBounds(N_i)                          ▷ From evaluation function
SortChildren(r) → N_1, ..., N_b                     ▷ By decreasingly optimistic
while (L := max_i L_i) < U_2 do                     ▷ r is assumed to be a MAX node
    if L_1 < L or H.UseProveBest(r) then s := 1
    else s := 2
    t_s := min[U_2, L_s + σ · (U_s − L_s)]
    τ_s := max[L, U_s − σ · (U_s − L_s)]
    MID(N_s, t_s, τ_s, ε) → L_s, U_s
    SortChildren(r) → N_1, ..., N_b
```

This procedure initiates the depth-first B* (df-B*) search at the root node r. df-B* searches an adversarial or non-adversarial tree with an evaluation function to provide lower and upper bounds on the game-theoretical value. The search terminates when the optimal next move at the root has been determined.

Algorithm 3 MID$(n, t, \tau, \epsilon) \rightarrow (L', U')$

```
L, U := GetBounds(n)                                ▷ Passed on from previous call
if L ≥ t or U ≤ τ or L = U then
    SaveBounds(n, L, U)                             ▷ Put in Transposition Table
    return (L, U)
N_i := CreateChildren(n)
SaveBounds(n, L, U)                                 ▷ Cycle avoidance (Nagai, 2002)
while true do
    if L ≥ t or U ≤ τ or L = U then
        SaveBounds(n, L, U)
        return (L, U)
    L_i, U_i := GetBounds(N_i)                      ▷ Retrieve from TT or compute anew
    SortChildren(n) → N_1, ..., N_b                 ▷ By decreasingly optimistic
    Discard(N_i where L_i > U or U_i < L)           ▷ Prune irrelevant nodes
    if IsMinimising(n) then
        t_1 := min[t, L_2 + ε · (U_1 − L_1)]        ▷ If no N_2 present, t_1 = t
        MID(N_1, t_1, τ, ε) → L_1, U_1
        min_i L_i, min_i U_i → L, U
    else if IsMaximising(n) then
        τ_1 := max[τ, U_2 − ε · (U_1 − L_1)]        ▷ If no N_2 present, τ_1 = τ
        MID(N_1, t, τ_1, ε) → L_1, U_1
        max_i L_i, max_i U_i → L, U
```

This procedure performs a multiple iterative deepening B* (Berliner, 1979) search with thresholds t and τ, inspired by df-PN (Nagai, 2002) and the $1 + \epsilon$ trick by Pawlewicz and Lew (2007). Each call of MID only returns when its thresholds are met. The thresholds and node selection procedure are chosen to search the tree similarly to B*, with some leniency controlled by the σ and ϵ parameters.

Replacement Schemes. The pseudocode given in the previous subsection makes use of a transposition table (TT), which requires a replacement scheme for different-node collisions (Nagai, 2002). Let n be the node to save to the TT, and $m \neq n$ be a node with the same key in the TT, that has been stored earlier. Our default replacement scheme for df-B*, in this thesis, is to always replace m with n. However, this may not be optimal. Other possible replacement schemes for df-B* may prioritise saving the node that is more likely to be reused, or saving the node that required the most effort to compute. Because df-B* does not search the tree with depth limits, a measure like maximum search depth is not very informative here. It may be better to use the total number of expansions or the size of the frontier as a measure of effort expended. However, the depth of a node can instead be used to determine how likely it is to be reused, as lower-depth nodes are closer to the root and may be checked more frequently. However, a specific difficulty with depth-first traversing B* is that we cannot determine, based on a TT entry alone, whether m is still globally relevant to the search or not. Even if we know that m required more effort to compute than n, it can still be better to save the less costly node n, especially if m, or one of its ancestors, has since become irrelevant to the search. The simplest solution is therefore to always replace m with n, which is what we apply in this thesis.

Note on Transpositions. The best-first search methods B* and B^{*2} are effective for searching on game trees, because they store the game tree in memory directly. However, many games have a natural structure which does not follow a tree. Rather, the structure of many games is that of a directed, often acyclic, graph (DAG). This means that, when structuring the game as a tree, the same position may be represented multiple times throughout, which we refer to as transpositions of the same game position. This usually results in additional search effort, exploring the same nodes multiple times, even if none are discarded during the search. On the contrary, df-B* is a depth-first search technique utilising a transposition table, which, as the name suggests, allows df-B* to store transpositions of game positions more efficiently (Nagai, 2002). However, the domain we apply in this thesis does not include any transpositions (see Section 4.2). As a result, df-B* may be shown to perform better, relative to B* and B^{*2}, on other domains.

On the Consideration of Bounds. The node selection for B* follows best-first logic based on evaluation function values, which are not always equally informative. For df-PN, the next node selected is always the node with the best lower bound for the amount of work needed to complete the search. The selected node in B* is not exactly a parallel for the most-proving node in PNS, as B* does not propagate information about required effort or work to the top of the tree. The result is that the node selected by B*, and by extension df-B*, is not the node that would lead to the fastest proof overall, as opposed to df-PN (Nagai, 2002). Palay (1983) realised this problem and noted that future research should develop a measure for the amount of work to be done in subtrees to enhance node selection for PB*. Such a measure would benefit the node selection of df-B* as well, and is an interesting candidate for future research.

Final Note on Precision. When implementing df-B* to the domain of artificial game trees, we had to consider the precision of the evaluation function supplied to df-B*. For evaluation functions with low granularity, relative to the size of the game tree, B* may have the tendency to spend a lot of search effort in very deep sub-trees. This is a result of the problem discussed in the previous paragraph. However, this tendency is heightened by df-B*, as it may keep searching a branch even after its root values have changed. To counteract this, we increased the Initial Range R (see Section 4.3) of our artificial game trees to provide higher granularity, and thus more precision, to the evaluation function values. Therefore, we advice the use of evaluation functions with high precision to reduce this effect, wherever possible.

Chapter 4

Artificial Game Trees

Search techniques are often applied to games in research due to the advantages that a game environment provides. The implementation of search techniques in a specific game environment may still require some additional complexity, however. This is especially the case when strong heuristics are required for algorithms to perform well in the chosen environment. The techniques being explored in this thesis are heuristic-dependent, so picking the right environment is important for a fair comparison of methods. We therefore apply the domain of Artificial Game Trees (AGTs) described by Berliner (1979), and later adapted by Anema (2025).

The structure of this chapter is as follows. First Section 4.1 describes the existing literature we build on for our implementation of AGTs. Second, Section 4.2 defines the domain of AGTs as it is applied in this thesis. Finally, Section 4.3 gives an overview of the parameters controlling the structure and generation of AGTs.

4.1 Origin

As opposed to testing the performance of our search techniques on a game environment, we do so on a computer-generated set of problems in the form of AGTs. Our implementation of AGTs is based on the description by (Anema, 2025), which is based on the description by Berliner (1979).

Berliner (1979) described the generation of canonical AGTs for the evaluation of the B* search technique. These trees were expanded during the search, and form the same structure regardless of the order in which the tree is explored. This was achieved by assigning a unique name and corresponding seed S_n to each node based on its position. Node values were generated such that expansion only ever tightens the bounds of node, as was required for B*. As a result, each tree has a unique solution and could potentially be infinitely large. Two parameters control the structure of the tree: the width, or branching factor, b, and the range R. However, these trees are all quite similarly structured, and the parameters do not allow for a lot of control.

Anema (2025) expanded on this model to evaluate B* and Disprove-Best B* on larger and more varied trees. These trees were generated in the same way as by (Berliner, 1979), but include two additional parameters: the distribution parameter k, and the growth factor g. This allowed for more control on the structure and size of the AGTs. The AGTs we describe in this thesis are based on the *AdjustAfter* variant as described by Anema (2025). However, we redefine how the seeds S_n for nodes are chosen and introduce another parameter: the relevance chance r.

4.2 Description

This section describes how AGTs are initialised and constructed in this thesis. The section is structured as follows. First Subsection 4.2.1 describes how we generate canonical AGTs, and how this differs from previous work. Second and finally, Subsection 4.2.2 describes the generation of AGTs with corresponding pseudocode.

4.2.1 Seeds for Canonical Trees

For the generation of canonical trees, Berliner (1979) assigned names to nodes in the tree based on their position. This approach assumes the tree has a fixed or maximum Branching Factor b. If we assign the name '0' to the root node, the name of a node n with parent p is chosen as:

$$\text{name}\,(n) = b \cdot \text{name}\,(p) + i \tag{4.1}$$

where $i = 1, \ldots, b$ is the index of n within the list of children of p. All generated values, centred on n, are then determined with a pseudorandom number generator (RNG) with a seed based on the name of n. The modification we introduce is in how this seed is chosen. The novel idea is the Initial Seed parameter S_0 and the node-specific seed S_n computed as:

$$S_n = \text{name}\,(n) + (\text{name}\,(n) + 1) \cdot S_0 \tag{4.2}$$

where the Initial Seed S_0 is a tree parameter which is best chosen as a large random 64-bit integer not equal to 0. Conceptually, S_0 is not only the tree specific seed, but also the node-specific seed S_{n_0} of the root node n_0 where $\text{name}\,(n_0) = 0$. We found that this approach resulted in fewer overlapping seeds than the approach described by (Berliner, 1979).

4.2.2 Node Generation

As described by Berliner (1979) and Anema (2025), only the root node n_0 of a tree T is provided initially, and the rest of the tree is generated dynamically. We apply the generation step for AGTs at the 'compute all possible moves' step of the search, which we refer to as node expansion. For every node expansion of any node n, it is ensured that if the bounds of n are updated after expansion, that they (a) never get wider, and (b) always have a non-zero chance to shrink.

Relevance Chance. The procedure for generating nodes is mostly identical to the *AdjustAfter* procedure described by Anema (2025). However, we introduce a minor modification. The novel idea is the introduction of the Relevance Chance parameter r, which relates to how children are generated when the Growth Factor g is greater than 1. Without r ($r = 0$), a large growth factor does not strictly increase the size of the tree, but may also decrease it. Children can be generated outside of the bounds of their parent, which makes them immediately irrelevant to the search. The Relevance Chance r is the chance of randomly forcing a node to be relevant to the search. This is applied to every generated node, and may have no effect if the node had already been relevant before. Preliminary experiments showed that, if $g \gg 1$ and k is relatively large, then r has a very strong effect on the size of the tree, with values closer to 1 making the tree increasingly harder to solve.

Pseudocode. The pseudocode of Algorithm 4 details the procedure of generating the children $c_1, \ldots, c_b$ of the node n given the tree-specific parameters S_0, b, k, g, and r. The Initial Range parameter R is defined as the initial upper bound of the root node n_0, where the lower bound is set to 0. Additionally, trees may be generated as adversarial or non-adversarial, where non-adversarial trees only have maximising nodes, and adversarial trees alternate between maximising and minimising nodes by depth. The root node n_0 is always maximising. Finally, as described by Berliner (1979), the lower and upper bounds of all nodes are represented by integer values, so a node can be generated as a terminal node if its lower bound is equal to its upper bound.

Algorithm 4 Expand(n, S_0, b, k, g, r) $\rightarrow c_1, \ldots, c_b$; the children of n

if Lower(n) = Upper(n) **then return** $\emptyset$
set seed $S_n \coloneqq \text{Name}(n) + (\text{Name}(n) + 1) \times S_0$ ▷ "$\sim_n$" indicates use of this generator
for all i in $1, \ldots, b$ **do**
  L $\coloneqq$ Lower(n)
  U $\coloneqq$ Upper(n)
  $\Delta \coloneqq (g-1) \times (U - L)$
  $D \coloneqq \lfloor \Delta \rfloor + I \sim_n I \in \{0, 1\} \;\; s.t. : \mathbb{E}(D) = \Delta$ ▷ gives $\Delta \in \mathbb{R}$ and $D \in \mathbb{Z} \approx \Delta$
  if isMaximising(n) **then**
    $v_1, \ldots, v_k \sim_n \mathcal{U}\{L - D, U\}$
  else
    $v_1, \ldots, v_k \sim_n \mathcal{U}\{L, U + D\}$
  if $g > 1$ **and** $r > X \sim_n \mathcal{U}[0, 1)$ **then**
    $v_{k+1} \sim_n \mathcal{U}\{L, U\}$
  $c_i \coloneqq$ child of n where:
    isMaximising(c_i) $\coloneqq$ **not** isMaximising(n) ▷ **if** tree is non-adversarial **then** $\coloneqq$ 'yes'
    Name(c_i) $\coloneqq$ Name(n) $\times\, b + i$
    Lower(c_i) $\coloneqq \min_j v_j$
    Upper(c_i) $\coloneqq \max_j v_j$
if $\min_i [\text{Lower}(c_i)] < \text{Lower}(n)$ **or** $\max_i [\text{Upper}(c_i)] > \text{Upper}(n)$ **then**
  $\ell \sim_n \mathcal{U}\{1, b\}$
  $v'_1, \ldots, v'_k \sim_n \mathcal{U}\{L, U\}$
  Lower(c_ℓ) $\coloneqq \min_j v'_j$
  Upper(c_ℓ) $\coloneqq \max_j v'_j$

Note on Transpositions. AGTs, as described in this chapter, differ from natural game trees in the presence of transpositions. Natural game trees often have an underlying structure that does not match an actual tree, but is rather a directed, and often acyclic, graph (DAG). This allows for multiple paths to reach the same node, referred to as a transposition. However, AGTs are strictly trees, with effectively no transpositions at all. This has no significant effect on search techniques that assume the game is strictly a tree. Although this may increase the relative difficulty of trees for techniques designed for DAGs. This includes depth-first search utilising transposition tables, like df-B* described in Section 3.3, but excludes the best-first search techniques B* and B*^{2}.

4.3 Parameter Overview

This section provides an overview of all tree-specific parameters for the generation of AGTs as described in this thesis. Table 4.1 lists all tree-specific parameters and their description. Table 4.2 lists the possible and recommended values for each parameter, and the expected effect of each parameter on the complexity of the tree. In this context, complexity refers to the number of node expansions needed, on average, by B* to solve the resulting trees.

Parameter		**Description**
Adversarial		Alternates maximising and minimising nodes
Initial Seed	S_0	Determines the result of random outcomes
Initial Range	R	Initial upper bound of the root, with a lower bound of 0
Branching Factor	b	Number of children generated at every internal node
Distribution Factor	k	Number of $v_1, \ldots, v_k$ considered for the generation of the bounds of a node where $Low(n) = \min_i (v_i)$ and $Upp(n) = \max_i (v_i)$
Growth Factor	g	Ratio of maximum allowed child range to parent range
Relevance Chance	r	Chance to generate an extra v_{k+1} of child within parent range which forces the child to be relevant to the search

Table 4.1: Parameters for the generation of artificial game trees, and their description. The additional value generated by the relevance chance r is on top of the number of values generated as determined by k. Therefore, the effective k (# values considered for bounds) is $k + r$.

Parameter		**Possible Values**	**Recommended Values**	**Effect on Complexity**
Adversarial		yes, no	any	uncertain
Initial Seed	S_0	64-bit integers	randomised, $\neq 0$	none
Initial Range	R	$n \geq 1 \in \mathbb{N}$	10^2 to 10^9	uncertain
Branching Factor	b	$n \geq 2 \in \mathbb{N}$	2 to 100	low
Distribution Factor	k	$n \geq 2 \in \mathbb{N}$	2 to 20	high
Growth Factor	g	$x > 0 \in \mathbb{R}$	≥ 1 < 1 shrinks tree	depends on k and r negative (if < 1)
Relevance Chance	r	$x \in [0, 1]$	< 1, if $g \gg 1$ any, otherwise	very high (if $g \gg 1$) none (if $g \leq 1$)

Table 4.2: Parameters for the generation of artificial game trees and their effects on the complexity (# nodes needed to solve problem) of the trees generated. Positive effect means increasing this parameter increases the complexity of the tree. The growth factor g has a high effect if k is also chosen to be large, and a negative effect if $g \gg 1$ while $r \approx 0$ and k is small.

Chapter 5

Experiments and Results

This chapter describes the performed experiments and their results, and is structured as follows. Section 5.1 describes the experimental setup, including hardware specifications, a list of terms, and experimental details of two experiments. Section 5.2 presents the experimental results.

5.1 Experimental Setup

This section describes the experiments that were performed for this thesis. First, hardware and software specifications for the experiments are provided in Subsection 5.1.1. Second, a list of terms is provided in Subsection 5.1.2. Third, Subsection 5.1.3 describes Experiment 1. Finally, Subsection 5.1.4 describes Experiment 2.

5.1.1 Hardware Specifications

The code for all search techniques, artificial game trees, and experiments was written and executed in Java 17. The individual runs of all experiments were run in parallel, with up to 32 runs computed simultaneously, on an AMD 5950X CPU with 64GB of DDR4 RAM running Windows 11. All runtime measurements were performed with the `java.time.Instant.now()` and `java.time.Duration.toNanos()` methods.

5.1.2 List of Terms

We define the following terms to describe the experimental methodology.

Tree refers to an Artificial Game Tree (AGT), as described in Chapter 4, which is uniquely defined by its parameters. The Initial Seed parameter S_0 is set to a unique value for each tree, as it determines the seeds of the pseudo-random number generators used to generate the tree structure.

Initialisation refers to the construction of the root of a tree, which is defined by its parameters. This tree can then be expanded indefinitely using any B* search technique to compute its solution.

Solution refers to the next best move, or 'best node', as computed from the root of a tree. The solution of each tree is not known in advance, but can be computed through B* search. A solution is found and proven whenever separation (Berliner, 1979) is reached at the root of the tree. Separation occurs when, assuming a node is maximising (without loss of generality), the lower bound of the best node exceeds or equals the upper bound of all alternative nodes. As an additional check, the solutions of all solved runs were cross-referenced with each other per tree, which confirms that all algorithms arrived at the same unique solution for each tree.

Run refers to, given a tree T and some B* search technique B, the process of computing the solution of T using B. A run may be restricted by an expansion limit and a node limit, which each may cause the run to terminate early. A run will be marked as solved if a solution was found and proven; this can also happen when a run was terminated early. The results of a run additionally include several metrics which are recorded during the search.

Metrics refers to the measurements made for the experiments, which are recorded during runs. Given a run of some tree T and some search technique B, we determine the following metrics:

- **Expansion** occurs whenever B selects the next node n of T for which the children of n are not accessible to B. 'Expansions' or 'Number of Nodes Expanded' refers to the total number of expansions during the run. Expansion is the metric which considers the cost of computing the children of a node n. The children of n may have been computed before, but not be accessible to B, if they have been previously removed from memory.
- **Nodes** are the core representation of game positions of T during the search. The metric 'nodes' or 'maximum nodes' refers to the maximum number of nodes held in memory by B during the run. Nodes are recorded differently for df-B*, as its transposition table takes up constant memory. We measure the number of nodes for df-B* as the maximum number of nodes held in its execution stack during the run.
- **Time** is the number of seconds that have passed from the start of the run to its termination. The preparation time for initialising T and B is excluded. Preparation time for search techniques includes the creation of transposition tables for df-B*.

Expansion limit refers to an artificial limit imposed on each run to control the total runtime of the experiment. The expansion limit is an additional stop condition which terminates a run early when the number of expansions exceeds the specified limit. The expansion limit is described as the maximum number of expansions performed during a single run.

Node limit refers to an artificial limit imposed on a run to control the amount of memory available to the search. The node limit is an additional stop condition which terminates a run early when the number of nodes in memory exceeds the specified limit. The node limit is described as the maximum number of nodes held in memory during a run.

5.1.3 Experiment 1: General Comparison

This subsection describes the first experiment, which is aimed at providing a general comparison of B*, B*^{2}, and df-B* on the domain of artificial game trees. We performed 10 runs with two variants of B*, 10 runs with two variants of B*^{2}, and 3 runs with two variants of df-B* on each of 25,920 unique trees, for a total of 1,192,320 runs. The experiment is described in the following

order. First, the included algorithm variants are described. Second, the experimental set of trees is described. Finally, the imposed expansion limits and node limits are described.

Algorithm Variants. Experiment 1 includes two variants of B*, two variants of B*^{2}, and two variants of df-B*. These variants have been implemented as follows:

1. **B*** is a modified B* (Berliner, 1979) implementation as described in Section 3.1. The strategy selection of B* is performed with the *A***R-criterium** as described in Subsection 2.1.2, where A refers to considering all alternatives in the calculation of the strategy.

2. **B*-alt** is mostly the same as **B*** described above. However, the strategy selection is instead performed with the **Alternated** method, as described in Subsection 2.1.2.

3. **B*^{2}** is as described in Section 3.2, with the following parameters. The first-level search is performed with **B***. The second-level search is performed with **B*-alt**. The size limits of the second-level tree are determined by a logistic growth function, as described in Subsection 3.2.2, where $a = 5 \cdot 10^5$ and $b = 5 \cdot 10^4$. The second-level search uses the stop conditions **separation** and **size limit** (see Subsection 3.2.1) and applies the **full disposal** approach for bounds preservation (see Subsection 3.2.1).

4. **B*^{2}-alt** is mostly the same as **B*^{2}** described above, with the following differences. The first-level search is performed with **B*-alt**. The second-level search uses **Always-PB** (see Subsection 2.1.2) for strategy selection.

5. **df-B*** is as described by Algorithm 2 and Section 3.3. The df-B*-specific thresholding parameters are chosen as $\sigma = 0.1$ and $\epsilon = 0.3$. Strategy selection is performed with the *A***R-criterium**, same as **B***. An initial maximum depth of 10 is applied to the search, which is incremented by 1 every time it is exceeded. Upon reaching the maximum depth, the search is forcibly backed up to the root. As an implementation detail, the search may also be forced to back up to the root if some operation-system-dependent stack-size limit is exceeded. Upon which the stack-size limit is doubled, and the search is resumed.

6. **df-B*-alt** is mostly the same as **df-B*** described above. However, the strategy selection is instead performed with the **Alternated** method, as described in Subsection 2.1.2.

Trees. Experiment 1 is performed on a diverse set of artificial game trees. The trees for this experiment were initialised with all combinations of the parameter values shown in Table 5.1, for a total of $6 \cdot 3 \cdot 6 \cdot 6 = 648$ tree variants. We did not include non-adversarial trees in the experiment, because their inclusion resulted in much smaller trees, which did not contribute much to the overall results. For each tree variant, we initialised 40 unique trees by applying a different Initial Seed S_0 (see Table 4.1) to each tree, for a total of $648 \cdot 40 = 25{,}920$ trees. The details of seed allocation are described in Section B.1 in the Appendix. The choice of seeds ensures that every tree is an independent problem with a unique solution.

Imposed Limitations. Each of the 1,192,320 runs was performed with an expansion limit of 5,000,000 and a node limit of $2^{20} = 1{,}048{,}576$. This node limit was realised for df-B* through the use of a transposition table of size 2^{20} (Subsection 3.3.2). The index for the transposition table of any node n from the artificial game tree is chosen as the 20 rightmost bits of a pseudo-randomly generated 64-bit value, seeded with the node-specific seed S_n (Section 4.2).

Parameter		Assigned Values
Adversarial		yes
Initial Range	R	10,000,000
Branching Factor	b	2, 3, 5, 10, 15, 30
Distribution Factor	k	4, 5, 6
Growth Factor	g	1, 1.1, 1.2, 1.3, 1.4, 1.5
Relevance Chance	r	0, 0.2, 0.3, 0.4, 0.5, 0.6

Table 5.1: List of parameter values for the 648 different tree variants included in Experiment 1. See Table 4.1 and Chapter 4 for the definitions and use of these parameters, and see Section B.1 for details on how the Initial Seed parameter S_0 was assigned for this experiment.

5.1.4 Experiment 2: Memory Usage

This subsection describes the second experiment, which explores the memory footprint of B*^{2} compared to B* by conducting runs with various node limits. The experimental setup was fine-tuned based on the results of Experiment 1. We performed 25 runs with two variants of B*^{2} and 10 runs with one variant of B* on each of 12,000 unique trees, for a total of 720,000 runs. The experiment is described in the following order. First, the included algorithm variants are described. Second, the experimental set of trees is described. Finally, the imposed expansion limits and node limits are described.

Algorithm Variants. Experiment 2 includes one variant of B* and two variants of B*^{2}. df-B* was not included in this experiment due to time constraints. However, its memory footprint is already well enough understood from the results of Experiment 1. The selected algorithm variants have been implemented as follows:

1. **B*** is the default B* implementation we describe for Experiment 1 in Subsection 5.1.3.
2. **B*^{2}** is the default B*^{2} implementation we describe for Experiment 1 in Subsection 5.1.3.
3. **B*^{2}-max** is mostly the same as **B*^{2}**. However, the second-level search is performed with a more lenient tree-size limit. Instead of a logistic growth function, the size of the second-level tree is limited only by the current size of the first-level tree (see Subsection 3.2.2).

Other variants from Experiment 1, **B*-alt** and **B*^{2}-alt**, were not included because their performance was too similar to **B*** and **B*^{2}**, based on the results of Experiment 1. **B*^{2}-max** was included because its memory footprint differed greatly from **B*^{2}** in preliminary experiments.

Trees. The artificial game trees for this experiment were selected based on the results of Experiment 1. Trees were initialised with all combinations of the parameter values shown in Table 5.2, for a total of $3 \cdot 2 \cdot 4 = 24$ variants. These variants were chosen as a subset of the 648 tree variants of Experiment 1 with roughly the highest proportion of trees requiring more than 50,000 node expansions but not exceeding the node or expansion limits. For each tree variant, we initialised 500 unique trees by providing a different Initial Seed S_0 (see Table 4.1) to each tree, for a total of $24 \cdot 500 = 12{,}000$ trees. Seeds are chosen such that none overlap with the seeds for Experiment 1. Further details of seed allocation are shown in Table A.1 in the Appendix. The choice of seeds ensures that every tree is an independent problem with a unique solution.

Parameter		Assigned Values
Adversarial		yes
Initial Range	R	10,000,000
Branching Factor	b	5, 10, 15
Distribution Factor	k	5, 6
Growth Factor	g	1.2, 1.3, 1.4, 1.5
Relevance Chance	r	0.4

Table 5.2: List of parameter values for the 24 different tree variants included in Experiment 2. See Table 4.1 and Chapter 4 for the definitions and use of these parameters, and see Table A.1 for details on how the Initial Seed parameter S_0 was assigned for this experiment.

Imposed Limitations. Unlike B*, B*^{2} behaves differently depending on the node limit. B*^{2} increasingly restricts the extent of its second-level searches as the first-level tree approaches the overall node limit. Therefore, we select various node limits to more precisely measure the memory requirements of B*^{2}. The 25 different node limits applied to **B***^{2} and **B***^{2}**-max** are listed in Table 5.3. For all B* runs, we apply the same node limit of 15,000,000. All runs also had an expansion limit of 10,000,000.

5	10	25	50	100	200	500	1000	2000
5000	10,000	15,000	20,000	50,000	75,000	115,000	150,000	200,000
275,000	350,000	450,000	600,000	700,000	850,000	1,000,000		

Table 5.3: List of 25 node limits applied to the B*^{2} runs of Experiment 2.

5.2 Results

This section describes the results from the experiments described in Section 5.1. All intervals shown in this section, denoted by '±', are 95% confidence intervals rounded to two significant digits. Additionally, all averages shown are rounded to the same decimal place as their interval half-widths. This section is structured as follows. Subsection 5.2.1 shows the results from Experiment 1. Subsection 5.2.2 shows the results from Experiment 2.

5.2.1 Experiment 1

The results from Experiment 1 include the recorded metrics of 1,192,320 runs on 25,920 trees, which were performed as described in Subsection 5.1.3. Recorded metrics include the number of expansions, the number of nodes, and the duration of runs, as defined in Subsection 5.1.2. Additionally, each run was recorded as 'solved' if separation was reached, and as 'unsolved' otherwise. In total, 1,005,723 runs were marked as solved, which accounts for 23,521 trees. Out of these trees, 15,131 trees were solved by all algorithm variants in all runs, which we refer to as 'the set of **universally solved trees**'. Figure 5.1 shows the number of trees solved by each of the six algorithm variants, which shows that **B***^{2} solved the most trees (23,477), and **df-B*** solved the least trees (15,309). However, this result depends on the choice of expansion and node limits. If we lower the expansion limit to 1,000,000, then **B***^{2} still solves the most trees (22,724), which is only 0.6% more than B* (22,586), which remains unchanged. If we instead lower the node limit to 300,000, then **B*** only solves 21,305 trees, while **B***^{2} remains unchanged.

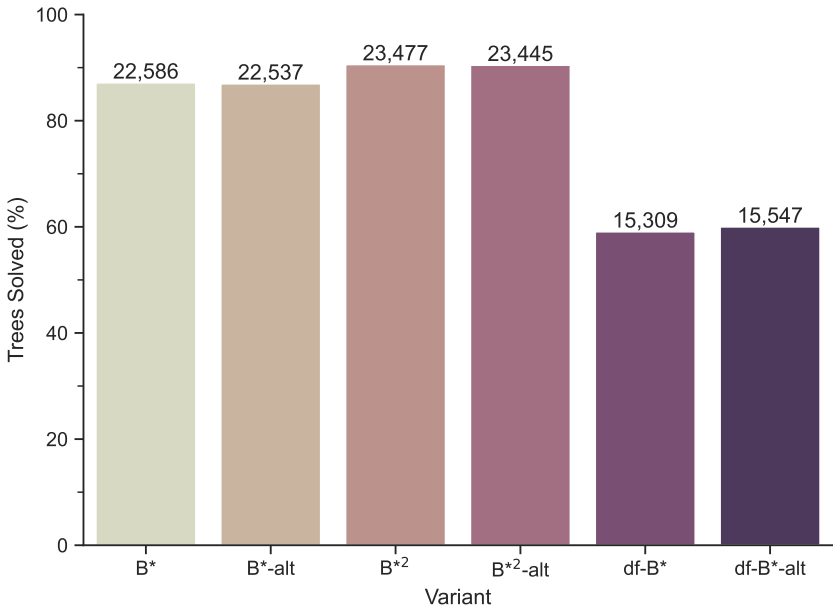


Figure 5.1: This figure shows the percentage and number of trees, out of 25,920, that were solved at least once, grouped by algorithm variant. The values atop each bar show the exact number of trees, whereas the height of the bar shows the percentage.

Efficiency. We compare the computational efficiency of these algorithm variants by measuring their relative performance on the same set of trees. Figure 5.2 shows the average number of expansions on the set of 15,131 universally solved trees, grouped by algorithm variant. It is shown that **df-B*** expands the most nodes ($\approx 10{,}200 \pm 2100$), and **B*** expands the least nodes ($\approx 539 \pm 23$). The results show no significant difference between **B*** and **B*-alt** ($\approx 562 \pm 24$), or between **B*^{2}** ($\approx 843 \pm 35$) and **B*^{2}-alt** ($\approx 854 \pm 36$). However, **df-B*-alt** ($\approx 5400 \pm 1200$) expands significantly fewer nodes than **df-B***. Taking the best result for each of the three B* techniques, we calculate that **B*^{2}** expanded on average 56% more nodes than **B***, whereas **df-B*-alt** expanded on average 902% more nodes than **B***.

In addition to the total number of expansions, we compute the number of expansions per second. Figure 5.3 shows the average ratio of the number of expansions to the run duration in seconds, grouped by algorithm variant. These rates were computed on a subset of all 25,920 trees. Due to an implementation detail, we could not accurately measure the duration of runs shorter than 0.0005 seconds; therefore, all trees with runtimes below 0.0005 seconds were excluded, resulting in 13,936 trees. The rate of expansion may indicate the level of overhead introduced by the algorithm variant. The results show that **B*-alt** has the highest rate of expansion ($\approx 205{,}390 \pm 580$), which is marginally higher than **B*** ($\approx 200{,}590 \pm 560$). Marginally lower is the expansion rate of **B*^{2}** ($\approx 194{,}630 \pm 500$), which is not significantly different from **B*^{2}-alt** ($\approx 195{,}600 \pm 510$). Finally, **df-B*** ($\approx 36,880 \pm 260$) and **df-B*-alt** ($\approx 37,750 \pm 260$) have the lowest rates of expansion. These results indicate that **B***, **B*-alt**, **B*^{2}**, and **B*^{2}-max** have a similar amount of computational overhead, whereas **df-B*** and **df-B*-alt** have significantly more overhead.

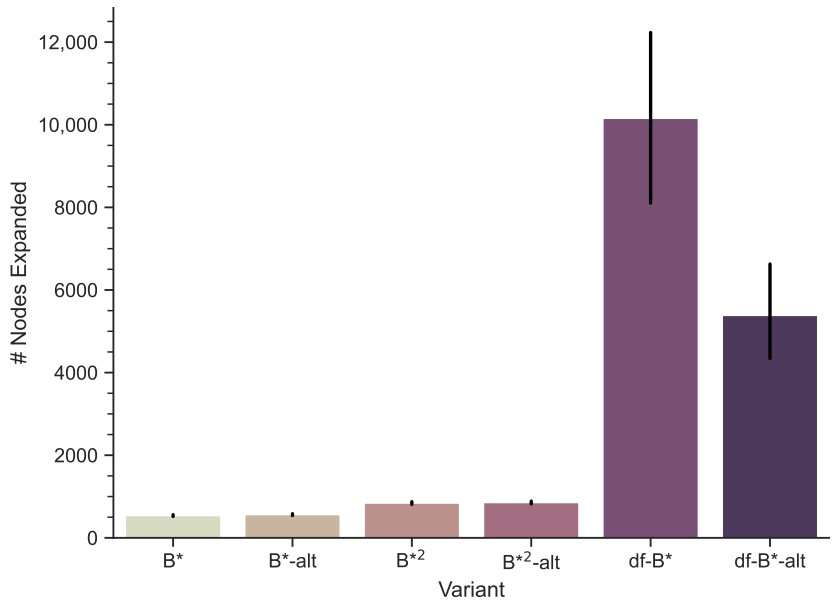


Figure 5.2: This figure shows the average number of nodes expanded by each of six algorithm variants on the set of 15,131 universally solved trees. Also shown are 95% confidence intervals.

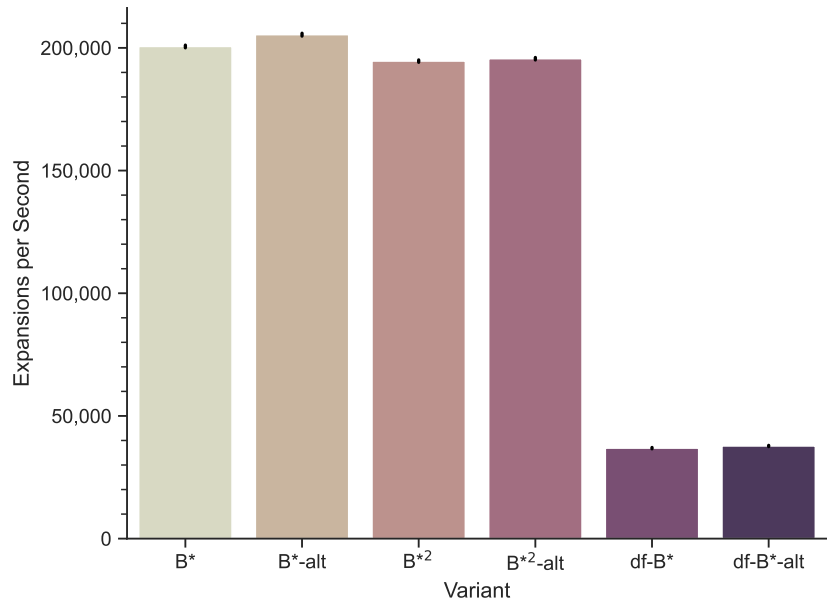


Figure 5.3: This figure shows the average number of nodes expanded per second of runtime (average of ratios), grouped by algorithm variant. Shown are all runs on 13,936 trees for which no runtime was less than 0.0005 seconds. Also shown are 95% confidence intervals.

Memory Usage. We compare the memory usage of different algorithm variants. Averaged over all 1,192,320 runs, as shown in Figure 5.5, we found that **B*** ($\approx 181{,}100 \pm 4400$) and **B*-alt** ($\approx 184{,}400 \pm 4500$) stored the most nodes in memory. Whereas **B*^{2}** ($\approx 37{,}830 \pm 990$) and **B*^{2}-alt** ($\approx 38{,}250 \pm 1000$) stored significantly fewer nodes in memory. Additionally, we recorded the maximum number of nodes on the execution stack for **df-B*** ($\approx 4130 \pm 83$) and **df-B*-alt** ($\approx 3910 \pm 80$). However, the memory usage of **df-B*** and **df-B*-alt** is effectively constant, as both variants always keep a transposition table in memory, with a size of $2^{20} = 1{,}048{,}576$ nodes, as described in Subsection 5.1.3. Figure 5.4 shows the relationship between expansions and nodes stored in memory, grouped by B* (**B*** and **B*-alt**), B*^{2} (**B***^{2} and **B***^{2}**-alt**), and df-B* (**df-B*** and **df-B*-alt**). The algorithm variants have been grouped by their core technique because there was no visible difference between the grouped variants. The graph shows how the three B* techniques handle memory differently. Among all solved runs, we find that B* stored at most 1,048,526 nodes in memory, whereas B*^{2} stored at most 281,270 nodes in memory, and df-B* stored at most 10,955 nodes in the stack. Zooming in on B*, we find that the rate of nodes stored per expansion is correlated with the branching factor ($r \approx 0.87$), which is an expected result.

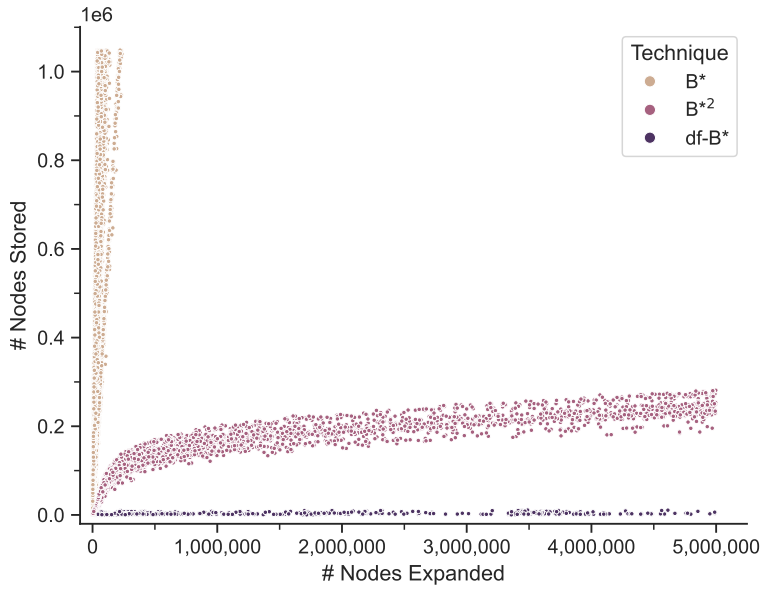


Figure 5.4: This figure shows the maximum number of nodes stored in memory, during a run, plotted against the number of nodes expanded, grouped by search technique. Only the 1,005,723 solved runs are shown. Values for df-B* are measured as the maximum number of nodes stored in the stack during the search, whereas the majority of df-B* memory usage is constant and determined by the size of the transposition table ($2^{20} = 1{,}048{,}576$).

Extra. In addition to these results, Table A.2 in the Appendix shows the relative performance of the six algorithm variants in terms of expansions, nodes stored, and run duration.

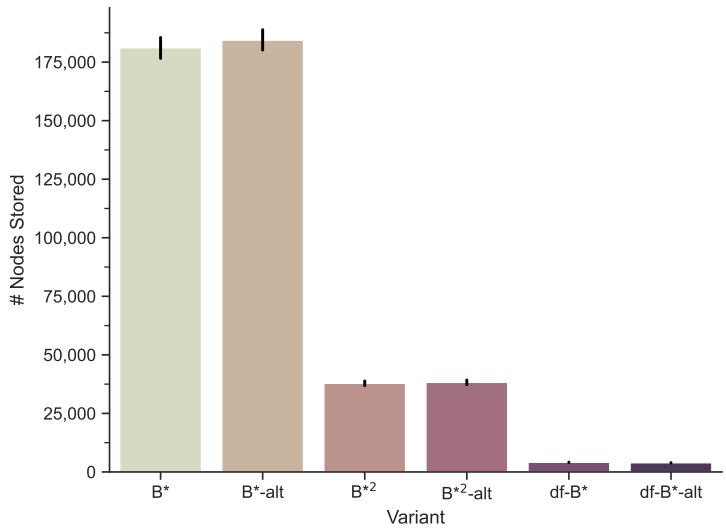
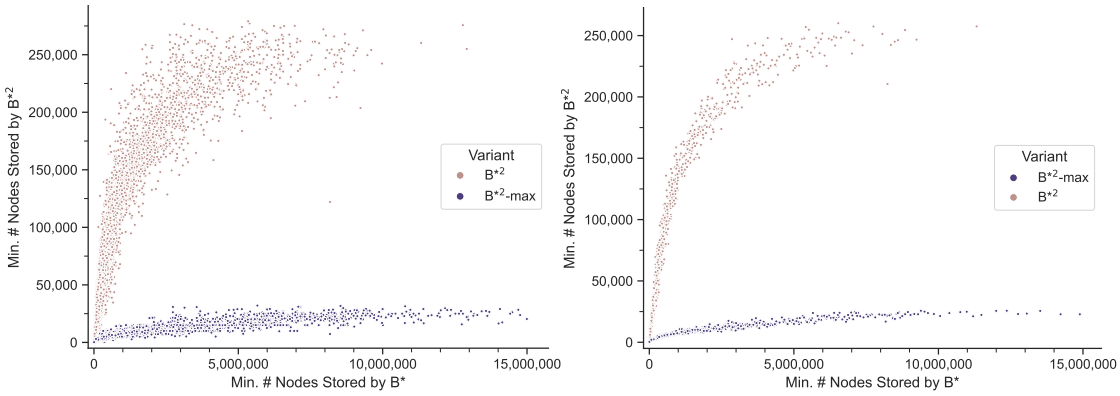


Figure 5.5: This figure shows the average number of nodes maximally stored in memory by each algorithm variant. All 1,192,320 runs are included. Values for df-B* are measured as the maximum number of nodes stored in the stack during the search.

5.2.2 Experiment 2

The results from Experiment 2 include the recorded metrics of 720,000 runs on 12,000 trees, as described in Subsection 5.1.4. With these results, we explore the memory usage of B^{*2} in more detail. For this experiment, we look at one key metric: the number of nodes stored in memory. We also record, for each run, whether it was 'solved' or 'unsolved', based on separation at the root node. We analyse the results mostly on a per-tree basis. For each tree, the results include 10 runs of **B***, 25 runs of $\mathbf{B^{*2}}$, and 25 runs of $\mathbf{B^{*2}}$**-max**. In this subsection, unless stated otherwise, the number of nodes stored by an algorithm variant on a tree refers to the minimum among all runs on that tree, excluding unsolved runs. For an initial comparison, we look at the set of 8733 trees which were solved at least once by all variants. Here, we find that $\mathbf{B^{*2}}$ ($\approx 54{,}800 \pm 1600$) stored significantly more nodes in memory, on average, than $\mathbf{B^{*2}}$**-max** ($\approx 3720 \pm 100$). Whereas **B*** ($\approx 617{,}000 \pm 27{,}000$) stored the most nodes in memory on average. The following results go into further detail.

Figure 5.6 shows the main result of this experiment: the memory footprint of B^{*2}. We directly compare the memory usage of **B*** to the memory usage of $\mathbf{B^{*2}}$ and $\mathbf{B^{*2}}$**-max**, where memory usage is represented in terms of the minimum number of nodes needed to solve a tree. The one-on-one data, including points for each of the 12,000 trees, is shown in Figure 5.6a. Here, some variance in algorithm performance on various trees is visible. We can see that $\mathbf{B^{*2}}$ has a sub-linear memory footprint relative to **B***'s memory usage. This relationship is shown more clearly in Figure 5.6b. Each point is the average result of 5 trees, adjacent in terms of **B***'s memory usage on those trees. We can also fit a line to this pattern. Fitting a $y = a\sqrt{x}$ line, where y is $\mathbf{B^{*2}}$'s memory usage and x is **B***'s memory usage, results in a good fit overall ($a = 114 \rightarrow R^2 \approx 0.92$), but overshoots greatly where $x > 500{,}000$ ($a = 116 \rightarrow R^2 \approx 0.58$). Figure 5.7 looks at the same result, but zoomed in on $\mathbf{B^{*2}}$**-max**. However, fitting the same line to this plot gives a more consistent result. The line $y = a\sqrt{x}$ fits well overall ($a = 7.6 \rightarrow R^2 \approx 0.92$), and still fits for $x > 500{,}000$ ($a = 7.6 \rightarrow R^2 \approx 0.76$). This line $y = 7.6\sqrt{x}$ is displayed on top of the results from Experiment 2 in Figure 5.7, which shows that the data visually matches it.



(a) Ungrouped: each dot corresponds to one tree (b) Grouped: each dot corresponds to 5 trees

Figure 5.6: This figure shows the memory footprint of **B*^{2}** and **B*^{2}-max** relative to **B*** on the set of 9087 trees solved by **B*** and at least one other variant. On the **left (a)**, the X-axis shows the minimum number of nodes stored in memory by **B*** out of 10 runs performed on the same tree. The Y-axis shows the minimum number of nodes stored in memory by **B*^{2}** or **B*^{2}-max** out of 25 runs performed on the same tree, only including solved runs. The **right (b)** figure shows the averages of 5 adjacent trees, ordered by X-axis value, for the plotted X and Y values in Figure (a). Thus, each dot corresponds to 50 runs of **B*** and 125 runs of **B*^{2}** or **B*^{2}-max**.

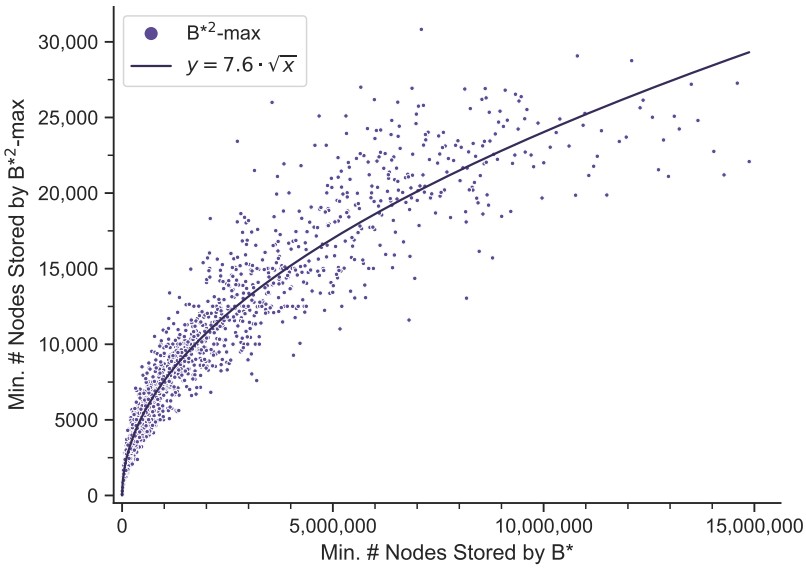


Figure 5.7: This figure shows the memory footprint of **B*^{2}-max** relative to **B*** on the set of 9080 trees solved by both variants. The X-axis shows the average over 2 trees of the minimum number of nodes stored in memory by **B*** out of 10 runs performed on the same tree. The Y-axis shows the average over 2 trees of the minimum number of nodes stored in memory by **B*^{2}-max** out of 25 runs performed on the same tree, only including solved runs. Thus, each dot corresponds to 20 runs of **B*** and 50 runs of **B*^{2}-max**. Additionally, a fit-line for the function $y = a \cdot \sqrt{x}$, with $a = 7.6$ is plotted. The coefficient of determination of the fitted line is $R^2 \approx 0.92$

Extrapolation. Observe the line fit for **B*^{2}-max** in Figure 5.7: $y = 7.6 \cdot \sqrt{x}$. It describes the amount of memory used by **B*^{2}-max** relative to **B***. But what does this relationship mean for the size of trees **B*^{2}-max** can solve? Let N be the maximum size tree **B*** can solve, in terms of number of nodes in memory. Then, inverting this equation, and assuming it can be extrapolated, we get that **B***2**-max** may be able to solve trees with up to $\approx 0.017 \cdot N^2$ number of nodes as searched by **B***. This is a very substantial increase, given the order of N^2. However, this result may be inconclusive as the fit excludes 207 trees unsolved by **B*^{2}-max** that were solved by **B***. The memory usage of **B*^{2}-max** for these trees is unknown, as they exceeded the expansion limit of the search.

Efficiency of B*^{2}-max Finally, there are some additional results comparing **B*^{2}** to **B*^{2}-max**. Figure 5.8 shows which percentage of all 12,000 trees each B*^{2} variant solved, where runs are grouped by their node limit. The figure shows that **B*^{2}-max** solves few to none new trees when the node limit is increased beyond 25,000, whereas **B*^{2}** reaches this point for node limits beyond 275,000. Beyond this point, the main limitation for both variants is the expansion limit rather than the node limit, which is 10,000,000 nodes for Experiment 2. It is also shown that **B*^{2}** solved fewer trees (8741) compared to **B*^{2}-max** (9080). Figure 5.9 shows the difference in the number of trees solved by **B*^{2}** and **B*^{2}-max** compared to the memory footprint of **B*** on these trees. This is a measure of algorithm efficiency rather than memory usage, because both **B*^{2}** and **B*^{2}-max** are primarily restricted by the expansion limit, as shown in Figure 5.8. Here, **B*^{2}-max** is shown to solve larger trees more effectively than **B*^{2}**. Additionally, on solved trees for which **B*** stored $\geq$ 10,000,000 nodes, 3 were solved by **B*^{2}** and 61 were solved by **B*^{2}-max**. Out of the trees unsolved by **B***, 1 was solved by **B*^{2}** and 32 were solved by **B*^{2}-max**, with at most ~32,000 nodes in memory, compared to ~250,000 for **B*^{2}**.

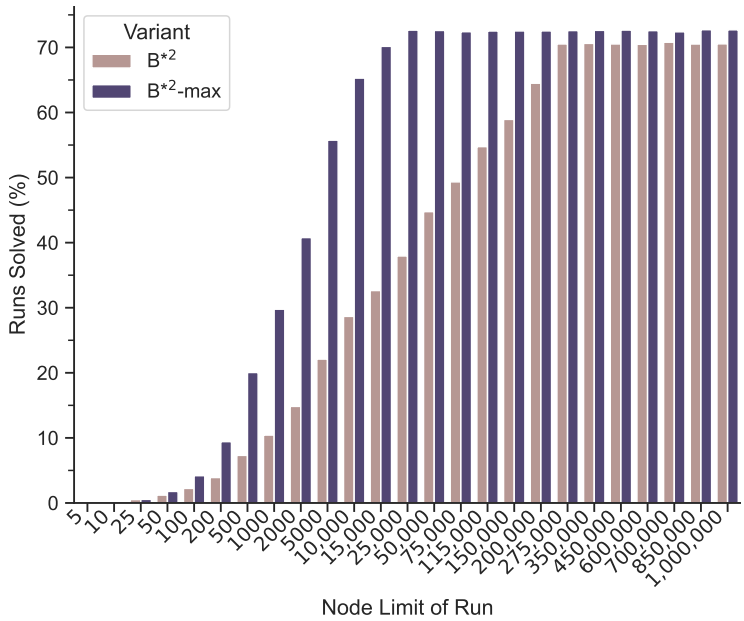


Figure 5.8: This figure shows the percentage of trees, out of the 12,000 included in Experiment 2, solved by **B*^{2}** or **B*^{2}-max** for each of the 25 different node limits imposed on the search. The X-axis of this figure is a list of all node limits and does not follow a linear scale.

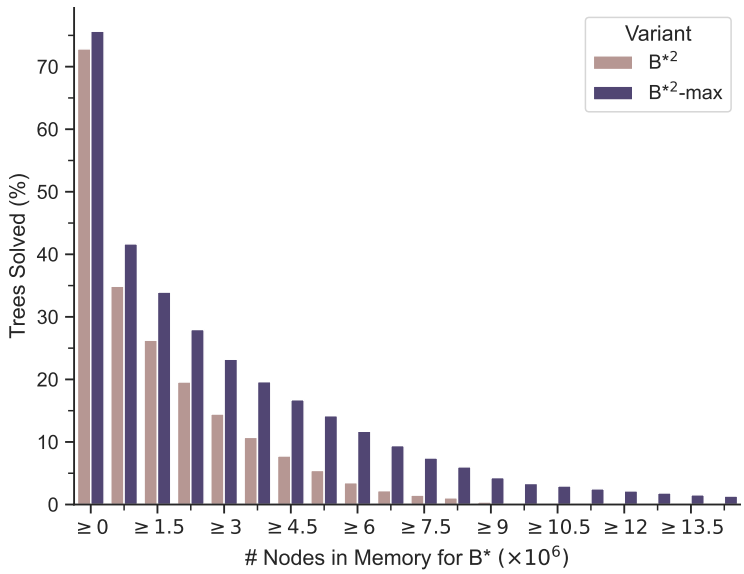


Figure 5.9: This figure shows, for each group of trees, the percentage that was solved by $\mathbf{B^{*2}}$ or by $\mathbf{B^{*2}}$**-max**. Each bar shows the group of trees, out of a total of 12,000, for which **B*** required at least the specified number of nodes in memory to solve. The leftmost bar includes all 12,000 trees from Experiment 2, whereas the rightmost bar includes only those trees for which **B*** stored $\geq$ 14,250,000 nodes in memory.

Chapter 6

Conclusions and Future Research

This chapter concludes the thesis. The research questions posed in the introduction are addressed in Section 6.1, and the problem statement is reassessed in Section 6.2. Finally, we propose ideas for future research in Section 6.3.

6.1 Research Questions

This section assesses the three research questions posed in Section 1.4 of the introduction.

1. *How can B* be adapted to multi-level (B^{*2}) or depth-first (df-B*) search?*

This question was posed to guide the design of the novel techniques introduced in this thesis. We draw inspiration from existing techniques, but there are several defining differences. The B^{*2} and df-B* techniques differ from the techniques they are based on, PN^2 (Breuker, 1998) and df-PN (Nagai, 2002), by the adaptations made to apply their concepts to the domain of B*: **(a)** B^{*2} differs from PN^2 by the nuance of strategy selection at each level of the search, and by the additional stop conditions for the second-level search. These adaptations are a natural result of adapting B* to multi-level search. **(b)** df-B* differs from df-PN by the separation between the root-level search and search on internal nodes. This is a natural result of strategy selection in B*. The separation between ϵ and σ is also a natural result, as inspired by Pawlewicz and Lew (2007), because df-B* may benefit from more frequent back-ups to the root, through reconsideration of the search strategy. Finally, df-B* reads values of solved nodes from the transposition table to prune them early, similar to df-PN. However, df-B* may also prune nodes that were not solved before, but have become irrelevant due to updates to other nodes. This is a natural result of node pruning in B* that is not present in Proof-Number Search (PNS) or df-PN.

2. *How do B^{*2} and df-B* affect the memory usage of search compared to B*?*

The main goal of developing B^{*2} and df-B* was to reduce the memory usage of B*. This research question aims to quantify the extent to which these techniques succeed in that goal. We first describe the resolution of this research question for B^{*2}, and then for df-B*.

B^{*2} is shown, in Figure 5.6, to require significantly less memory than B* to solve the same trees. This is highlighted in Subsection 5.2.1 as well as in Subsection 5.2.2. Table A.2 shows that, on the trees examined in Experiment 1, B^{*2} required between 15% and 17% of the number of nodes

in memory as B* to solve the same sets of trees. For Experiment 2, as described in Subsection 5.2.2, we found that, on the set of 8733 trees solved by both, B* stored on average 617,000 nodes and B^{*2} on average 54,800. That is a ratio of 9% on that specific set of trees. However, these averages on specific sets of trees are only indicative results. A more conclusive result is shown in Figure 5.6 and Figure 5.7. The latter of which approximately quantifies $y = 7.6\sqrt{x}$ ($R^2 \approx 0.92$) as the relationship describing the relative memory usage of B^{*2} (y) to B* (x). The inverse of this suggests that B^{*2}, given the same memory limitations as B*, may be able to solve trees of sizes up to $\approx 0.017 \cdot N^2$, where N is the memory usage of the maximum size trees solved by B*. This result assumes the maximum second-level tree-size limit is applied for B^{*2} (see Subsection 3.2.2). These results show that B^{*2} can solve larger trees than B*, but still has an upper limit of trees that can be solved based on the available memory. This aligns with existing research on multi-level search techniques. First, Baum and Smith (1997) hypothesised a square-root relationship in memory usage of a two-stage Bayesian Search, which matches our result. Second, Breuker (1998) similarly found significantly reduced memory usage of PN^2 compared to PNS, which we also observe for B^{*2}.

df-B* uses a transposition table and execution stack for memory. The transposition table utilises constant memory, whereas the execution stack stores at most $b \cdot d$ nodes, where b is the branching factor of the tree and d is the maximum depth of the search. The memory usage of the stack is included in our results, as shown in Figure 5.4 and Figure 5.5. Subsection 5.2.1 describes that df-B* stored at most 10,955 nodes in its execution stack compared to over a million nodes stored in memory for B*, on solved trees of Experiment 1. For the purpose of comparing the memory usage of df-B* to B*, df-B* only uses constant memory, as the memory usage of df-B* is in the order of $b \cdot d$ rather than the b^d for B*. This result is as expected; df-B* can search very large trees with effectively no limit on the size of trees that can be solved, if given enough time.

3. *How do B^{*2} and df-B* compare to B* in terms of computational efficiency?*

We compare the computational efficiency of B^{*2} and df-B* to B* to understand the cost of reduced memory usage. As to achieve this reduced memory usage, both techniques actively discard a large portion intermediate search results. This comes at a computational cost, which we mostly measure in the number of expansions performed during the search. We first describe the results for computational efficiency of B^{*2}, and then of df-B*.

B^{*2} surprisingly seems not to always give up on computational efficiency in exchange for reduced memory usage. Subsection 5.2.1 describes how, when the expansion limit was reduced to 1,000,000, which is less than the applied node limit, B^{*2} still solved 0.06% more trees than B*. This shows that B^{*2} may sometimes solve the same trees more efficiently than B* does. However, Subsection 5.2.1 and Figure 5.2 show that B^{*2} performed on average 56% more expansions than B* on the set of 15,131 universally solved trees from Experiment 1. Table A.2 also shows that B^{*2} expanded between 2.6 and 3 times as many nodes as B* on the same sets of over 22,000 trees. We conclude that the latter is likely a more realistic result, as the set of universally solved trees is biased by the reduced set of trees solved by df-B*.

Additionally, Figure 5.3 shows that B^{*2} has a comparable rate of expansion to B*, which corresponds to a similar computational overhead for the two techniques. As a result, the runtime of B^{*2} scales predictably with its increased number of expansions compared to B*.

Experiment 2 shows an additional result on the computational efficiency of B^{*2}. Figure 5.9 shows that, compared to the B^{*2} variant shown in Experiment 1, B^{*2} with the maximum tree-size limit more efficiently solves larger trees, despite neither variant ever terminating due to the

upper node limit of 1,000,000 nodes. This indicates that the computational efficiency of B^{*2} might be improved by modifying the tree-size limits of the second-level search.

df-B* is found to have a significantly reduced computational efficiency compared to B*, as seen in Figure 5.1, Figure 5.2, and Figure 5.3. Table A.2 shows that df-B* expands on average between 22 and 23 times as many nodes as B* on the same sets of trees. However, the results from Figure 5.3 show that df-B* also has significantly increased computational overhead compared to B* and B^{*2}. Similarly, Winands et al. (2004) found, for the implementation of the PNS variant PDS-PN, that the depth-first PDS search (Nagai, 1999) had a significant amount of computational overhead compared to PNS and PN^2. The computational overhead may therefore be a more systemic problem with depth-first game-tree solvers, although this would require additional research to confirm. Nevertheless, the computational efficiency of df-B* may be increased with further research into the technique. Our implementation of df-B* is not optimal. Compared to B* and B^{*2}, which have been modified to improve their performance on this domain, df-B* is implemented in this thesis as its minimum viable version. As such, there is likely room for improvement, at least in the number of expansions needed to solve the same trees.

6.2 Problem Statement

We reassess the problem statement of this thesis, which was formulated as follows:

How can B be extended to search on much larger problems?*

By applying inspirations from the memory-efficient PNS variants PN^2 and df-PN, B* can be extended to solve larger problems. Results on problems of up to 15 million nodes in size show that B^{*2} may solve problems of up to 0.017 times the square of the size of problems as solved by B*. The memory usage of B^{*2} is in the order of a square-root of the memory usage by B*. To search even larger problems, multiple levels of B* can be chained together for B^{*k}, which is expected to use memory in the order of a k-root of the memory used by B*. Additionally, the depth-first B* variant df-B* can effectively search any size trees, with near constant memory usage, if given enough time. Overall, we conclude that B^{*2} is a good alternative to B* when searching relatively large problems with limited memory, whereas df-B* is a good alternative to B* when searching very large problems of undetermined size.

6.3 Future Research

This section concludes this thesis by suggesting directions for future research.

First, we suggest that future research explore more domains to which B* can be applied. We find that B* does not have to be limited by memory, as B^{*2} works efficiently while reducing its memory usage, and df-B* can solve problems regardless of their size. This may allow for B* to be applied to domains that it was not applicable to before. B* does not need to be restricted to game-tree search domains. It may solve any problem which can be represented as a tree with lower and upper bounds on some value to be optimised. Therefore, B^{*2} and df-B* may expand the possibilities of B* by reducing or removing its memory limitation. Additionally, the domain we apply in this thesis is different from many other domains in that its underlying structure does not contain any transpositions (see Section 4.2). Whereas df-B* is expected to solve trees with transpositions more efficiently, as results from the use of a transposition table (see Subsection 3.3.2). As a result, other domains may show better performance of df-B*.

Second, future research could explore a novel idea to improve B* and its variants: (dis)proof numbers for value ranges. An interesting finding from our preliminary experiments was that B^{*2} seemed to perform much better, in all aspects, than B* on certain large trees. To combat this, we implemented two adjustments for B*, which are described in Section 3.1: a maximum to the number of rootless updates, and a random chance to select an alternative node at internal node selection. The effectiveness of the latter modification and the randomness of its function suggest that there may be more room for improvement in internal node selection, which is not a new idea (Palay, 1983; Berliner & McConnell, 1996). This problem has been addressed with probabilistic B*, but this approach lacks a metric for measuring the amount of work needed to progress on a sub-tree. Here we can take inspiration from PNS once again, and introduce the idea of (dis)proof numbers to B*. A (dis)proof number in B* can represent the minimum number of expansions required to (dis)prove the value of a node lies above a certain threshold. The main problem with this approach is that a range of (dis)proof numbers may be needed for each of multiple thresholds. However, as of the time of writing, this is a mostly unexplored concept with the potential to improve B*, B^{*2}, df-B*, and even PB* (Palay, 1983).

Third, future research could evaluate several enhancements to B^{*2}. This includes the stop conditions for shallow and deep irrelevance, and the four bounds preservation approaches mentioned in Subsection 3.2.1. Lazy evaluation may also be applied to B^{*2}, which we have not implemented in this thesis. Exploring the promise of these enhancements may give a fuller picture of the potential of B^{*2} and multi-level search in general.

Finally, we suggest future research explore the potential of df-B* further. Our findings show promise for df-B*, but also challenges. Our implementation of df-B*, as described in this thesis, is far from optimal. We expect that the computational efficiency of df-B* can be greatly increased by implementing a better internal node selection and by including a metric for required work on sub-trees. df-B* could also be improved with a more informed replacement scheme for the transposition table. Additionally, df-B* may have more potential on domains with many transpositions, as mentioned in Section 3.3.2.

References

Allis, L. V., van der Meulen, M., & van den Herik, H. J. (1994). Proof-Number Search. *Artificial Intelligence*, *66*(1), 91–124. doi: 10.1016/0004-3702(94)90004-3

Anema, P. R. (2025). *New Advances in B** (Tech. Rep.). Maastricht, the Netherlands: Department of Advanced Computing Sciences, Maastricht University. Retrieved from `https://github.com/Pascalos99/ReportNewAdvancesInBstar/blob/main/Anema2025.pdf`

Baum, E. B., & Smith, W. D. (1997). A Bayesian approach to relevance in game playing. *Artificial Intelligence*, *97*(1), 195-242. doi: 10.1016/S0004-3702(97)00059-3

Berliner, H. J. (1979). The B* tree search algorithm: A best-first proof procedure. *Artificial Intelligence*, *12*(1), 23–40. doi: 10.1016/0004-3702(79)90003-1

Berliner, H. J., & McConnell, C. (1996). B* probability based search. *Artificial Intelligence*, *86*(1), 97–156. doi: 10.1016/0004-3702(95)00092-5

Breuker, D. M. (1998). *Memory versus Search in Games* (PhD thesis, Department of Advanced Computing Sciences, Maastricht University, Maastricht, the Netherlands). doi: 10.26481/dis.19981016db

Campbell, M., Hoane Jr, A. J., & Hsu, F. (2002). Deep Blue. *Artificial Intelligence*, *134*(1), 57–83. doi: 10.1016/S0004-3702(01)00129-1

Coulom, R. (2007). Efficient selectivity and backup operators in Monte-Carlo Tree Search. In *Proceedings of the 5th Computers and Games Conference (GC'06)* (Vol. 4630, pp. 72–83). Berlin, Heidelberg, Germany: Springer. doi: 10.1007/978-3-540-75538-8_7

Franz, C., Mogk, G., Mrziglod, T., & Schewior, K. (2022). Completeness and diversity in depth-first Proof-Number Search with applications to retrosynthesis. In *Proceedings of the International Joint Conference on Artificial Intelligence* (Vol. 31, pp. 4747–4753). IJCAI. doi: 10.24963/ijcai.2022/658

Heifets, A., & Jurisica, I. (2012). Construction of new medicines via game proof search. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 26, pp. 1564–1570). Palo Alto, California, USA: AAAI Press. doi: 10.1609/aaai.v26i1.8331

Kishimoto, A., & Müller, M. (2005). Search versus knowledge for solving life and death problems in Go. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 20, pp. 1374–1379). Menlo Park, California, USA: AAAI Press / MIT Press.

Kishimoto, A., Winands, M. H. M., Müller, M., & Saito, J.-T. (2012). Game-tree search using proof numbers: The first twenty years. *ICGA Journal*, *35*(3), 131–156. doi: 10.3233/ICG-2012-35302

Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, *6*(4), 293–326. doi: 10.1016/0004-3702(75)90019-3

Nagai, A. (1999). *A New Depth-First Algorithm for AND/OR Trees* (Master thesis). Department of Information Science, University of Tokyo, Tokyo, Japan.

Nagai, A. (2002). *df-PN Algorithm for Searching AND/OR Trees and its Applications* (PhD thesis). Department of Information Science, University of Tokyo, Tokyo, Japan.

Palay, A. J. (1982). The B* tree search algorithm–new results. *Artificial Intelligence*, *19*(2), 145–163. doi: 10.1016/0004-3702(82)90034-0

Palay, A. J. (1983). *Searching with Probabilities* (PhD thesis, Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Pennsylvania, USA). Retrieved from `https://apps.dtic.mil/sti/html/tr/ADA221478/`

Pawlewicz, J., & Lew, Ł. (2007). Improving depth-first PN-Search: $1+\epsilon$ trick. In *Proceedings of the 5th Computers and Games Conference (GC'06)* (Vol. 4630, pp. 160–171). Berlin, Heidelberg, Germany: Springer. doi: 10.1007/978-3-540-75538-8_14

Schaeffer, J., Burch, N., Björnsson, Y., Kishimoto, A., Müller, M., Lake, R., Lu, P., & Sutphen, S. (2007). Checkers is solved. *Science*, *317*(5844), 1518–1522. doi: 10.1126/science.1144079

Schaeffer, J., Culberson, J., Treloar, N., Knight, B., Lu, P., & Szafron, D. (1992). A world championship caliber Checkers program. *Artificial Intelligence*, *53*(2), 273-289. doi: 10.1016/0004-3702(92)90074-8

Schaeffer, J., Lake, R., Lu, P., & Bryant, M. (1996). CHINOOK The world man-machine Checkers champion. *AI Magazine*, *17*(1), 21–29. doi: 10.1609/aimag.v17i1.1208

Seo, M., Iida, H., & Uiterwijk, J. W. H. M. (2001). The PN*-search algorithm: Application to Tsume-Shogi. *Artificial Intelligence*, *129*(1), 253–277. doi: 10.1016/S0004-3702(01)00084-4

Sheppard, B. (2002). World-championship-caliber Scrabble. *Artificial Intelligence*, *134*(1), 241–275. doi: 10.1016/S0004-3702(01)00166-7

Sheppard, B. (2003). Endgame play in Scrabble. *ICGA Journal*, *26*(3), 147–164. doi: 10.3233/ICG-2003-26302

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, *529*(7587), 484-489. doi: 10.1038/nature16961

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., & Hassabis, D. (2017). Mastering the game of Go without human knowledge. *Nature*, *550*(7676), 354-359. doi: 10.1038/nature24270

Tesauro, G. (1994). TD-Gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, *6*(2), 215-219. doi: 10.1162/neco.1994.6.2.215

Tesauro, G. (2010). TD-Gammon. In C. Sammut & G. I. Webb (Eds.), *Encyclopedia of Machine Learning* (pp. 955–956). Boston, Massachusetts, USA: Springer. doi: 10.1007/978-0-387-30164-8_813

Thomsen, T. (2000a). Lambda-search in game trees – with application to Go. *ICGA Journal*, *23*(4), 203–217. doi: 10.3233/ICG-2000-23402

Thomsen, T. (2000b, December 21). *Lambda-Search Java-Code (Version 2.0).* Retrieved 2026, January 18, from `https://www.t-t.dk/go/cg2000/code20.html`

Winands, M. H. M., Uiterwijk, J. W. H. M., & van den Herik, H. J. (2004). An effective two-level Proof-Number Search algorithm. *Theoretical Computer Science*, *313*(3), 511–525. doi: 10.1016/j.tcs.2002.10.006

Wu, I.-C., Lin, H.-H., Lin, P.-H., Sun, D.-J., Chan, Y.-C., & Chen, B.-T. (2011). Job-level Proof-Number Search for Connect6. In *Proceedings of the 7th Computers and Games Conference (GC'10)* (pp. 11–22). Berlin, Heidelberg, Germany: Springer. doi: 10.1007/978-3-642-17928-0_2

Wu, I.-C., & Lin, P.-H. (2010). Relevance-zone-oriented proof search for Connect6. *IEEE Transactions on Computational Intelligence and AI in Games*, *2*(3), 191–207. doi: 10.1109/TCIAIG.2010.2060262

Xu, C., Ma, Z. M., Tao, J., & Xu, X. (2009). Enhancements of Proof Number Search in Connect6. In *Chinese Control and Decision Conference* (Vol. 21, pp. 4525–4529). doi: 10.1109/CCDC.2009.5191963

Appendix A

Additional Tables

Branching Factor b	Distribution Factor k	Growth Factor g	Allocated Seeds S_0
5	5	1.2	0
		1.3	1
		1.4	2
		1.5	3
	6	1.2	4
		1.3	5
		1.4	6
		1.5	7
10	5	1.2	8
		1.3	9
		1.4	10
		1.5	11
	6	1.2	12
		1.3	13
		1.4	14
		1.5	15
15	5	1.2	16
		1.3	17
		1.4	18
		1.5	19
	6	1.2	20
		1.3	21
		1.4	22
		1.5	23

Table A.1: This table shows how the initial seed parameter S_0 of artificial game trees is allocated for Experiment 2, which is described in Subsection 5.1.4. Based on the parameter values of a tree, we assign the value s from 0 to 23, as read from the table above. Then the ranges of values S_0 for the trees included in the experiment are computed as $S_0 = 762391543368789717 + 24i + s$, where i are all integer values from 0 to 499 inclusive. As such, a unique seed is assigned to each tree, for the total of 12,000 trees included in Experiment 2.

Variant	Cost	Size	Time	Trees
B*	1	1	1	22,179
B*-alt	1.05	1.08	1.05	22,071
B*^{2}	2.70	0.16	2.29	22,177
B*^{2}-alt	2.99	0.17	2.69	22,176
df-B*	23.14	0.02	93.54	15,256
df-B*-alt	22.97	0.03	94.48	15,461

(a) Trees solved by **B***

Variant	Cost	Size	Time	Trees
B*	0.95	0.93	0.95	22,071
B*-alt	1	1	1	22,161
B*^{2}	2.72	0.15	2.38	22,157
B*^{2}-alt	2.64	0.16	2.28	22,159
df-B*	22.06	0.02	91.40	15,256
df-B*-alt	22.10	0.02	93.58	15,460

(b) Trees solved by **B*-alt**

Variant	Cost	Size	Time	Trees
B*	0.37	6.15	0.44	22,177
B*-alt	0.37	6.53	0.42	22,157
B*^{2}	1	1	1	23,048
B*^{2}-alt	1.05	1.04	1.10	22,965
df-B*	14.77	0.19	74.30	15,257
df-B*-alt	14.70	0.19	75.35	15,462

(c) Trees solved by **B*^{2}**

Variant	Cost	Size	Time	Trees
B*	0.33	5.82	0.37	22,176
B*-alt	0.38	6.36	0.44	22,159
B*^{2}	0.96	0.97	0.91	22,965
B*^{2}-alt	1	1	1	23,049
df-B*	14.52	0.18	73.51	15,257
df-B*-alt	14.51	0.19	74.95	15,462

(d) Trees solved by **B*^{2}-alt**

Variant	Cost	Size	Time	Trees
B*	0.04	41.35	0.01	15,256
B*-alt	0.05	44.21	0.01	15,256
B*^{2}	0.07	5.33	0.01	15,257
B*^{2}-alt	0.07	5.42	0.01	15,257
df-B*	1	1	1	15,257
df-B*-alt	0.53	0.87	0.67	15,132

(e) Trees solved by **df-B***

Variant	Cost	Size	Time	Trees
B*	0.04	39.31	0.01	15,461
B*-alt	0.05	41.35	0.01	15,460
B*^{2}	0.07	5.16	0.01	15,462
B*^{2}-alt	0.07	5.22	0.01	15,462
df-B*	1.89	1.15	1.49	15,132
df-B*-alt	1	1	1	15,462

(f) Trees solved by **df-B*-alt**

Table A.2: These tables show the relative performance of 6 B* variants, based on the results of Experiment 1, as described in Subsection 5.2.1. The "Cost", "Size", and "Time" columns show the ratios of the number of expansions, the number of nodes, and the duration of runs, relative to the variant in the highlighted row. Ratios of means are computed only over the set of trees which were solved by all iterations of both variants. The "Trees" column shows the number of trees in each set.

Appendix B

Additional Details

B.1 Seed Allocation Experiment 1

Subsection 5.1.3 describes how trees are initialised for Experiment 1. However, the values for the Initial Seed S_0 are determined for each tree individually. The seed allocation for trees in Experiment 1 uses a similar table to Table A.1. However, this table would be too large to include, as there are 648 unique parameter combinations for the trees of Experiment 1. Seeds are similarly assigned to Experiment 2, with the addition of the Relevance Chance parameter r (see Chapter 4). The Relevance Chance column is included on the right side of all other parameter columns in the table, in the same order as the columns of Table 5.3. Seeds are then computed as $S_0 = 6623708332940258033 + 648i + s$, where i are all integer values from 0 to 39 inclusive, and s is incrementally assigned from 0 to 647 to each of the 648 unique parameter combinations. The procedure thus assigns a unique seed to each of the 25,920 trees included in Experiment 1.

Appendix C

Additional Figures

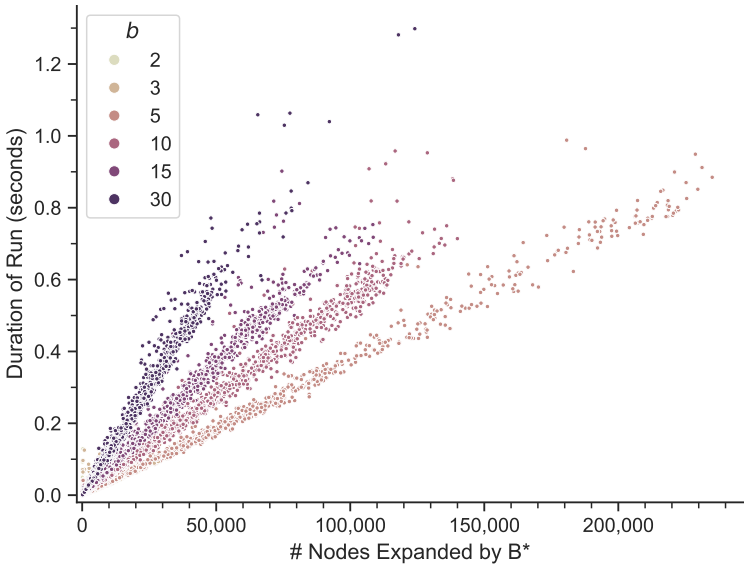


Figure C.1: This figure shows the duration of a run against the number of expansions for all runs solved by **B*** or **B*-alt**, from Experiment 1 (see Subsection 5.2.1). Runs displayed are categorised by the branching factor of the tree, which is negatively correlated with the number of expansions per second ($r \approx -0.54$).

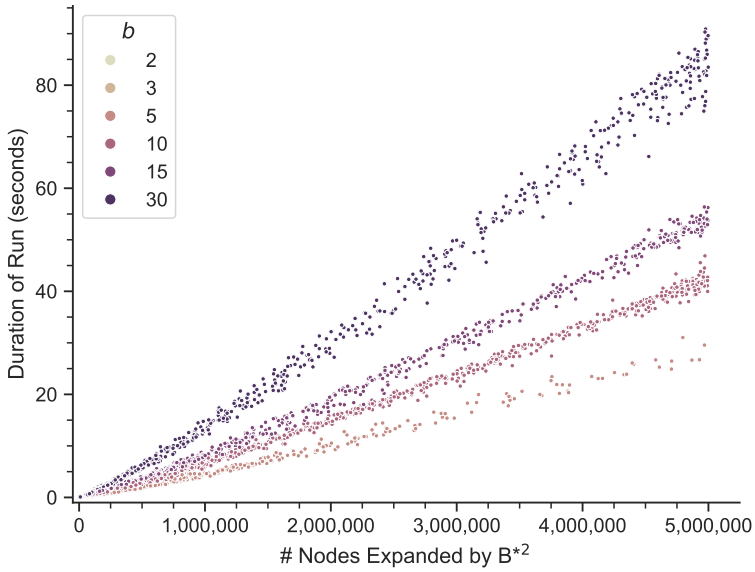


Figure C.2: This figure shows the duration of a run against the number of expansions for all runs solved by **B*^{2}** or **B*^{2}-alt**, for Experiment 1 (see Subsection 5.2.1). Runs displayed are categorised by the branching factor of the tree, which is negatively correlated with the number of expansions per second ($r \approx -0.62$).

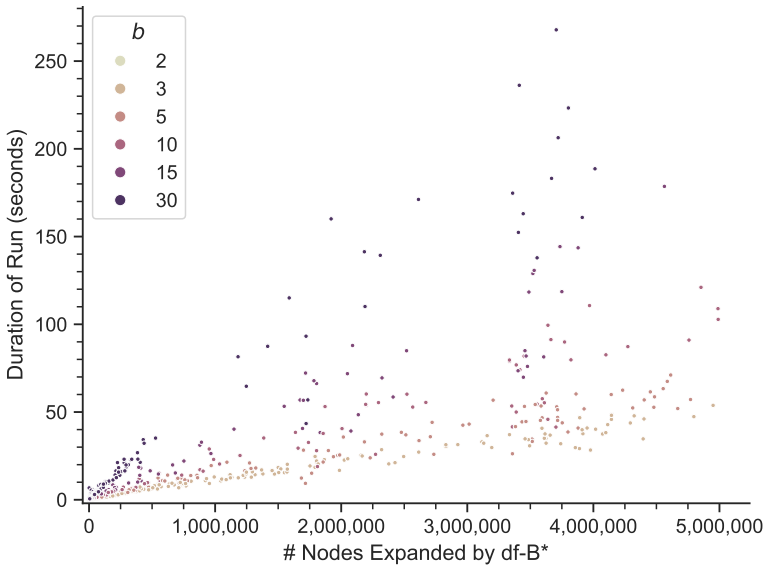


Figure C.3: This figure shows the duration of a run against the number of expansions for all runs solved by **df-B*** or **df-B*-alt**, for Experiment 1 (see Subsection 5.2.1). Runs displayed are categorised by the branching factor of the tree, which is not correlated with the number of expansions per second ($r \approx -0.03$).